Advanced Representation Learning
(고급표현학습)

Instructor: Donghyeon Cho

The Department of Computer Science

Hanyang University

한양대학교
HANYANG UNIVERSITY

Contents of Table

- 1. Pseudo Labeling
  - Label propagation
  - Self-Training
  - Reducing confirmation bias
- 2. Pseudo Labeling with Consistency Regularization
  - MixMatch
  - ReMixMatch
  - FixMatch
  - DivideMix

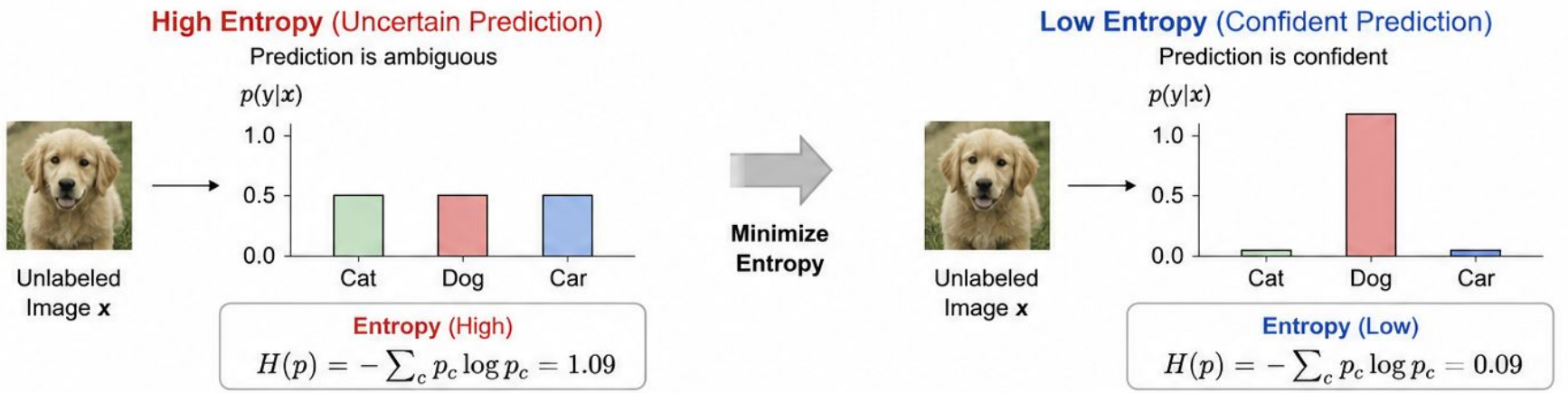
Pseudo Labeling

Pseudo Labeling

- Key Idea
  - **Predict labels** for unlabeled samples using the current model.
  - Treat **high-confidence** predictions as **pseudo labels**.
  - Train the model on **both labeled and pseudo-labeled** data.
- Advantages
  - Simple and easy to implement.
  - Scales well to large unlabeled datasets.
  - Foundation of many modern SSL methods.

Pseudo Labeling

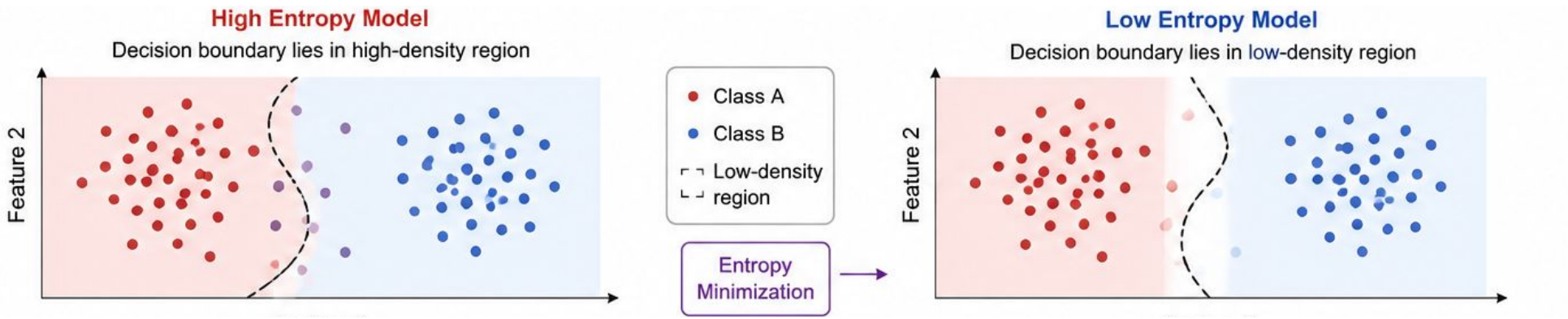
- Why Does Pseudo Labeling Work?
  - Pseudo Labeling is approximately equivalent to **Entropy Regularization**.
  - It encourages low-density separation between classes.
- Entropy Regularization?
  - High entropy → uncertain prediction
  - Low entropy → confident prediction
  - → Minimize the prediction entropy.



Bengio et al., NeurIPS 2004, Semi-supervised Learning by Entropy Minimization

Pseudo Labeling

- Why Does It Help?
  - Minimizing entropy moves decision boundaries toward low-density regions, producing better-separated clusters.



Bengio et al., NeurIPS 2004, Semi-supervised Learning by Entropy Minimization

Pseudo Labeling

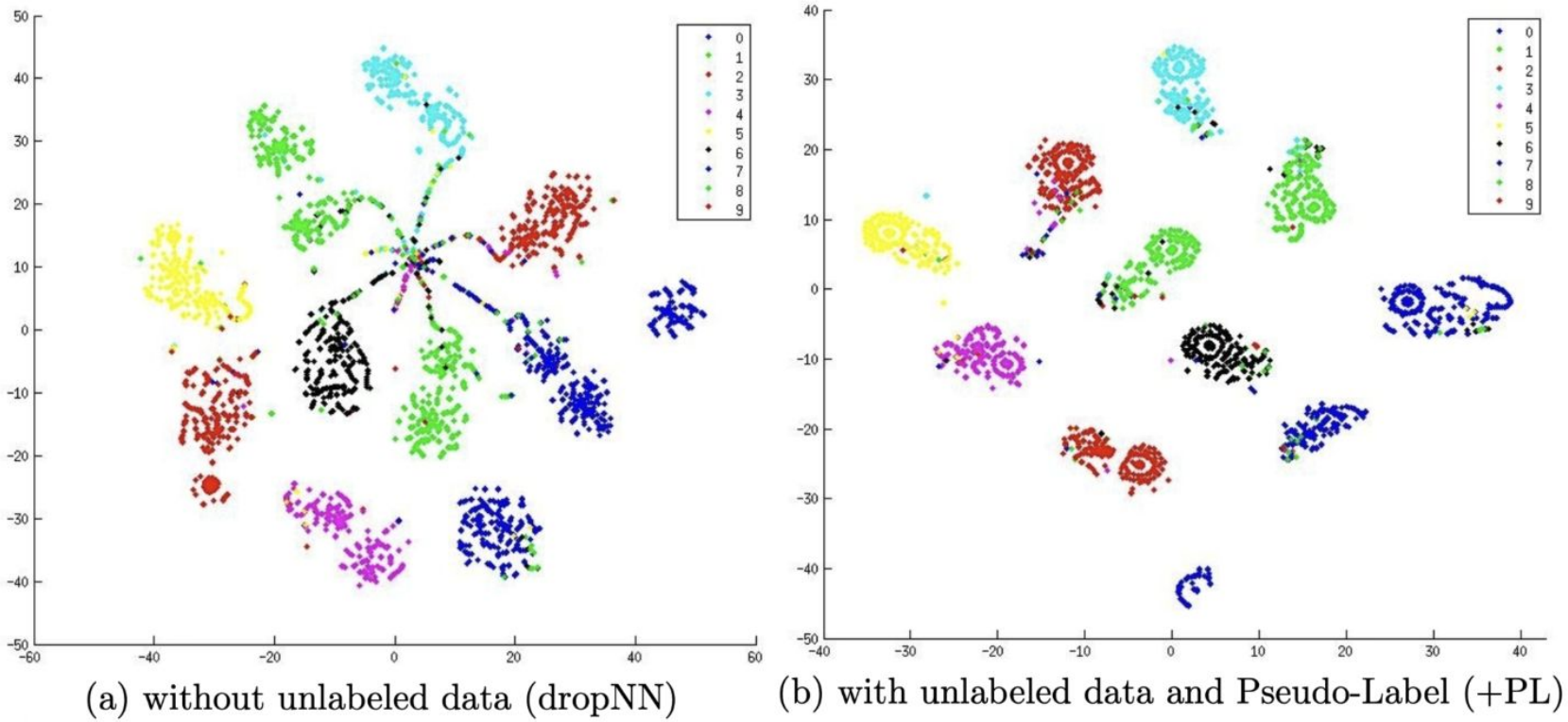
- Connection to Pseudo Labeling
  - Entropy Regularization encourages
    - (Once the prediction is sufficiently confident)
      - Soft prediction can be converted into a hard label.
      - ex) $p(y|x)=[0.96,0.03,0.01] \rightarrow y^*=\text{argmax}p(y|x)$

Pseudo Labeling can be viewed as the hard-label approximation of Entropy Regularization.

Lee, ICML 2013, Pseudo-Label : The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks

Pseudo Labeling

- Pseudo labeling leads to better segregation in the learned embedding space.

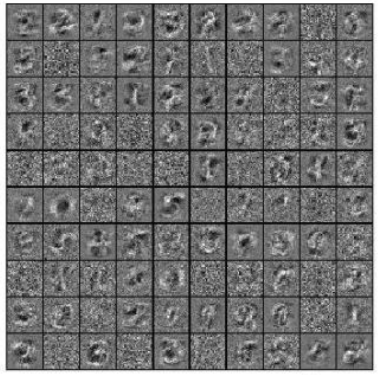


(a) without unlabeled data (dropNN) (b) with unlabeled data and Pseudo-Label (+PL)

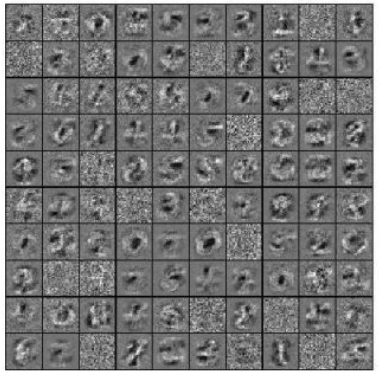
t-SNE visualization of outputs on MNIST test set by models training (a) without and (b) with pseudo labeling on 60000 unlabeled samples, in addition to 600 labeled data.

Lee, ICML 2013, Pseudo-Label : The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks

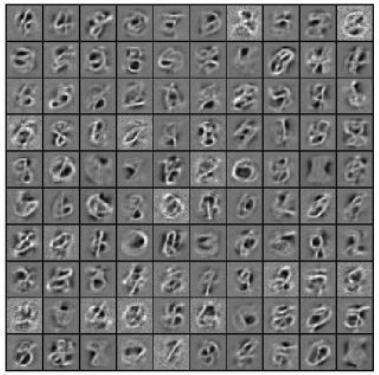
Pseudo Labeling



(a) without unlabeled data (dropNN)



(b) with unlabeled data and Pseudo-Label (+PL)



(c) using unsupervised pre-training with DAE (+PL+DAE)

Figure 2. Some filters on layer 1 of the network trained with or without unlabeled data on MNIST. 600 labeled data is used for supervised training.

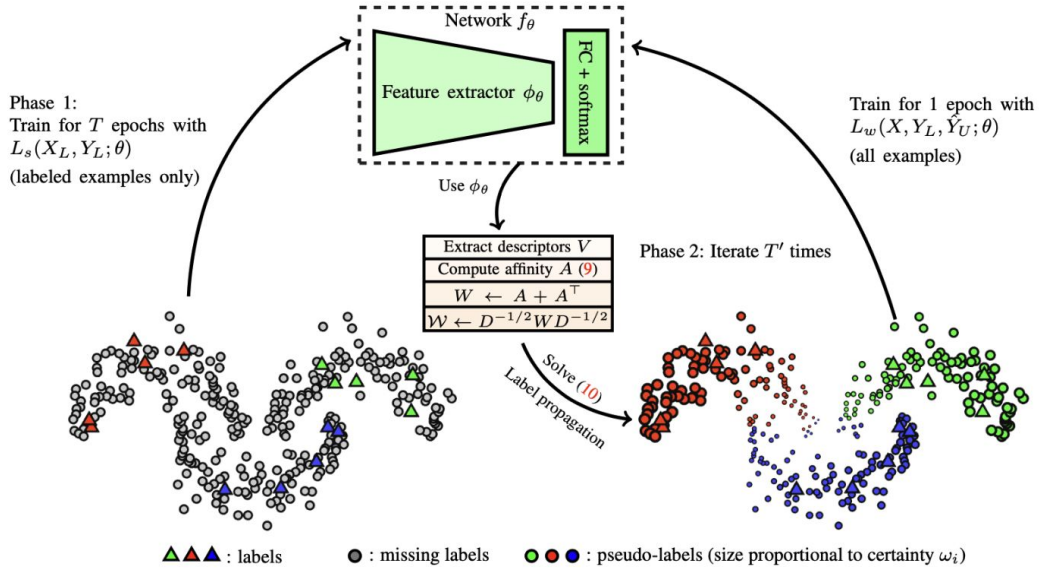
Table 2. Classification error on the MNIST test set with 600, 1000 and 3000 labeled training samples. We compare our method with results from (Weston et al., 2008; Rifai et al., 2011b). dropNN is our network model trained without unlabeled data, +PL with unlabeled data and Pseudo-Label, +PL+DAE using unsupervised pre-training with DAE in addition.

METHOD	100	600	1000	3000
NN	25.81	11.44	10.7	6.04
SVM	23.44	8.85	7.77	4.21
CNN	22.98	7.68	6.45	3.35
TSVM	16.81	6.16	5.38	3.45
DBN-RNCA	-	8.7	-	3.3
EMBEDNN	16.86	5.97	5.73	3.59
CAE	13.47	6.3	4.77	3.22
MTC	12.03	5.13	3.64	**2.57**
DROPNN	21.89	8.57	6.59	3.72
+PL	16.15	5.03	4.30	2.80
+PL+DAE	**10.49**	**4.01**	**3.46**	2.69

Lee, ICML 2013, Pseudo-Label : The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks

Label Propagation

- Key Idea
  - Construct a similarity graph in the feature space.
  - **Propagate** labels from **labeled samples to nearby unlabeled** samples.
  - Assign pseudo labels based on **neighborhood relationships**.
- How does it work?
  - Learn feature embeddings.
  - Build a similarity graph.
  - Propagate labels through graph connections.
  - Use propagated labels for training.



Iscen, et. al., CVPR 2019, Label Propagation for Deep Semi-supervised Learning

Label Propagation

Algorithm 1 Label propagation for deep SSL

procedure LPDSSL(Training examples X, labels Y_L)
  $\theta \leftarrow$ initialize randomly
  for epoch $\in [1, \ldots, T]$ **do**
    $\theta \leftarrow$ OPTIMIZE$(L_s(X_L, Y_L; \theta))$ ▷ mini-batch optimization
  end for
  for epoch $\in [1, \ldots, T']$ **do**
    for $i \in \{1, \ldots, n\}$ **do** $\mathbf{v}_i \leftarrow \phi_\theta(x_i)$ ▷ extract descriptors
    for $(i, j) \in \{1, \ldots, n\}^2$ **do** $a_{ij} \leftarrow$ affinity values (9)
    $W \leftarrow A + A^\top$ ▷ symmetric affinity
    $\mathcal{W} \leftarrow D^{-1/2} W D^{-1/2}$ ▷ symmetrically normalized affinity
    $Z \leftarrow$ solve (10) with CG ▷ diffusion
    for $(i, j) \in U \times C$ **do** $\hat{z}_{ij} \leftarrow z_{ij} / \sum_k z_{ik}$ ▷ normalize Z
    for $i \in U$ **do** $\hat{y}_i \leftarrow \arg\max_j \hat{z}_{ij}$ ▷ pseudo-label
    for $i \in U$ **do** $\omega_i \leftarrow$ certainty of $\hat{y}_i$ (11) ▷ pseudo-label weight
    for $j \in C$ **do** $\zeta_j \leftarrow (|L_j| + |U_j|)^{-1}$ ▷ class weight/balancing
    $\theta \leftarrow$ OPTIMIZE$(L_w(X, Y_L, \hat{Y}_U; \theta))$ ▷ mini-batch optimization
  end for
end procedure

Nearest neighbor graph. Given a network with parameters θ, we construct the descriptor set $V = (\mathbf{v}_1, \ldots, \mathbf{v}_l, \mathbf{v}_{l+1}, \ldots, \mathbf{v}_n)$, where $\mathbf{v}_i := \phi_\theta(x_i)$. A sparse *affinity matrix* $A \in \mathbb{R}^{n \times n}$ with elements

$$a_{ij} := \begin{cases} [\mathbf{v}_i^\top \mathbf{v}_j]_+^\gamma, & \text{if } i \neq j \wedge \mathbf{v}_i \in \mathrm{NN}_k(\mathbf{v}_j) \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

Label propagation. Estimating matrix Z by (6) is impractical for large n because the inverse matrix $(I - \alpha\mathcal{W})^{-1}$ is not sparse. We rather use the the conjugate gradient (CG) method to solve linear system

$$(I - \alpha\mathcal{W})Z = Y, \quad (10)$$

Iscen, et. al., CVPR 2019, Label Propagation for Deep Semi-supervised Learning

Label Propagation

Dataset	CIFAR-10			
Nb. labeled images	500	1000	2000	4000
Fully supervised	49.08 ± 0.83	40.03 ± 1.11	29.58 ± 0.93	21.63 ± 0.38
TDCNN [36]†	-	32.67 ± 1.93	22.99 ± 0.79	16.17 ± 0.37
Network prediction (1) + weights	35.17 ± 2.46	23.79 ± 1.31	16.64 ± 0.48	13.21 ± 0.61
Ours: Diffusion prediction (7) + weights	32.40 ± 1.80	22.02 ± 0.88	15.66 ± 0.35	12.69 ± 0.29
VAT [26]†	-	-	-	11.36
Π model [23]†	-	-	-	12.36 ± 0.31
Temporal Ensemble [23]†	-	-	-	12.16 ± 0.24
MT [38]†	-	27.36 ± 1.30	15.73 ± 0.31	12.31 ± 0.28
MT [38]	27.45 ± 2.64	19.04 ± 0.51	14.35 ± 0.31	11.41 ± 0.25
MT + Ours	$\mathbf{24.02 \pm 2.44}$	$\mathbf{16.93 \pm 0.70}$	$\mathbf{13.22 \pm 0.29}$	$\mathbf{10.61 \pm 0.28}$

Table 2. Comparison with the state of the art on CIFAR-10. Error rate is reported. "13-layer" network is used. The top part of the table corresponds to training with pseudo-labels, while the bottom part of the table includes methods that are complementary to ours, as shown by the combination of our method with MT. † denotes scores reported in prior work.

Iscen, et. al., CVPR 2019, Label Propagation for Deep Semi-supervised Learning

Label Propagation

- Compared with Pseudo Labeling

Pseudo Labeling	Label Propagation
Predict each sample independently	Exploit neighborhood relationships
Confidence-based	Similarity-based
Ignores data manifold	Leverages manifold structure

- Advantages
  - Produces smoother pseudo labels.
  - Better utilizes unlabeled data.
  - Exploits the cluster assumption.

Iscen, et. al., CVPR 2019, Label Propagation for Deep Semi-supervised Learning

Label Propagation

- Limitations
  - Requires constructing a similarity graph.
    - Expensive for large datasets.
  - Performance depends on feature quality.

Label Propagation improves pseudo labels by leveraging the geometric structure of the feature space rather than treating each sample independently.

Iscen, et. al., CVPR 2019, Label Propagation for Deep Semi-supervised Learning

Pseudo Labeling(Self-Training)

- Pseudo Labeling is naturally performed as an iterative teacher–student process.
- Teacher
  - Predicts pseudo labels for unlabeled samples.
  - Provides supervision for the student.
  - Often the previous model or an EMA model.
- Student
  - Learns from labeled data and pseudo labels.
  - Produces a better model after training.
  - Becomes the teacher in the next iteration.

As the model improves, the pseudo labels become more accurate, leading to further improvements.

Self-Training

- Key Idea
  - Train a model using labeled data.
  - Predict pseudo labels for unlabeled samples.
  - Add high-confidence predictions to the labeled set.
  - Repeat the process iteratively.
- Characteristics
  - **Iterative** learning framework.
  - Gradually **expands** the labeled dataset.
  - The teacher improves over iterations, producing better pseudo labels.

Nigam, et. al., CIKM 2000, Analyzing the Effectiveness and Applicability of Co-training

Self-Training

- Noisy Student (Xie et al., 2020)
  - Use the teacher to generate pseudo labels.
  - Train the student with additional noise.
  - The student becomes more robust than the teacher.
- Why Add Noise?
  - Noise prevents the student from simply copying the teacher.
  - Instead, it learns a more robust decision boundary.
    - RandAugment
    - Dropout
    - Stochastic Depth

Xie, et. al., CVPR 2020, Self-training with Noisy Student improves ImageNet classification

Self-Training

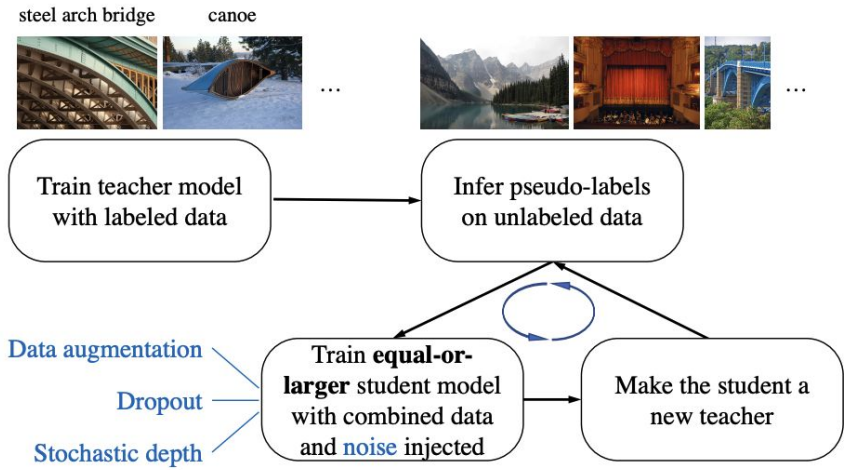


Figure 1: Illustration of the NoisyStudent method. (All shown images are from ImageNet.)

	ImageNet top-1 acc.	ImageNet-A top-1 acc.	ImageNet-C mCE	ImageNet-P mFR
Prev. SOTA	86.4%	61.0%	45.7	27.8
NoisyStudent	**88.4%**	**83.7%**	**28.3**	**12.2**

Table 1: Summary of key results compared to previous state-of-the-art models [84, 55]. Lower is better for mean corruption error (mCE) and mean flip rate (mFR).

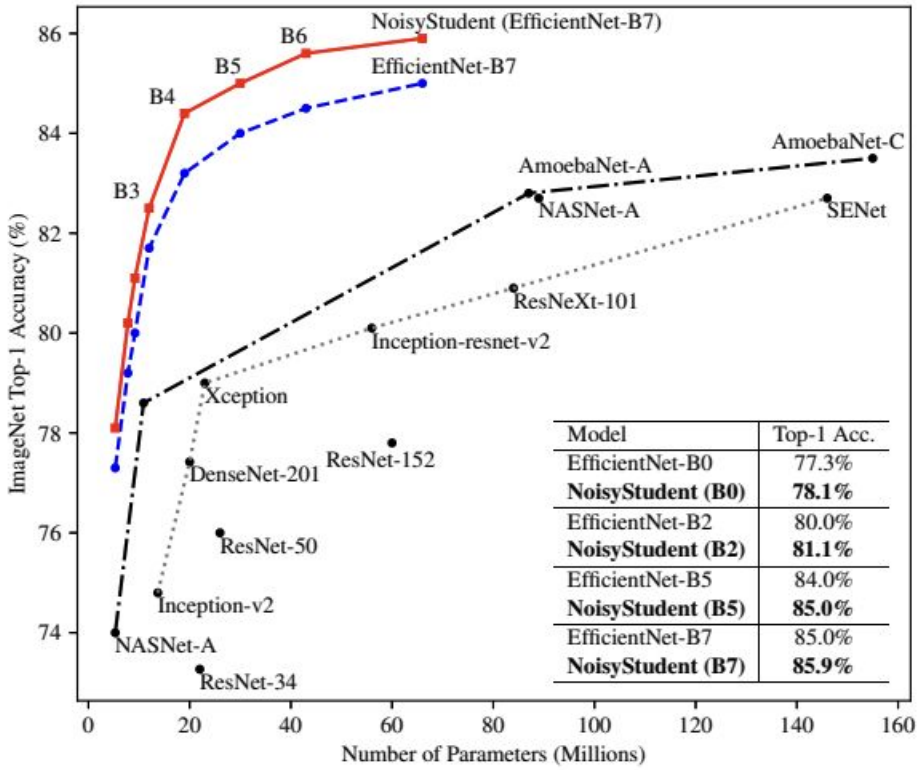


Figure 2: NoisyStudent leads to significant improvements across all model sizes for EfficientNet. We use the same architecture for the teacher and the student and do not perform iterative training.

Xie, et. al., CVPR 2020, Self-training with Noisy Student improves ImageNet classification

Self-Training

- Key Findings
  - Larger student > teacher
  - Soft pseudo labels > hard labels
  - Strong augmentation improves generalization
  - Achieved state-of-the-art ImageNet performance

Noisy Student extends self-training by making the student learn under stronger perturbations than the teacher, leading to progressively better models.

Xie, et. al., CVPR 2000, Self-training with Noisy Student improves ImageNet classification

Reducing Confirmation Bias

- Problem
  - Pseudo labels generated by an **imperfect teacher** can be incorrect.
  - The student is then trained to imitate these incorrect labels.
- Why is it harmful?
  - Incorrect pseudo labels **accumulate**.
  - The student reinforces the teacher's mistakes.
  - Performance gradually degrades.

Reducing Confirmation Bias

- How can we reduce it?
  - MixUp with soft labels
    - Given two samples, and their corresponding true or pseudo labels

$$\bar{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$$
$$\bar{y} = \lambda y_i + (1-\lambda) y_j \Leftrightarrow \mathcal{L} = \lambda [y_i^\top \log f_\theta(\bar{\mathbf{x}})] + (1-\lambda)[y_j^\top \log f_\theta(\bar{\mathbf{x}})]$$

  - Oversampling labeled data
  - Strong data augmentation
  - Dropout regularization

Arazo, et. al., IJCNN 2020, Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning

Reducing Confirmation Bias

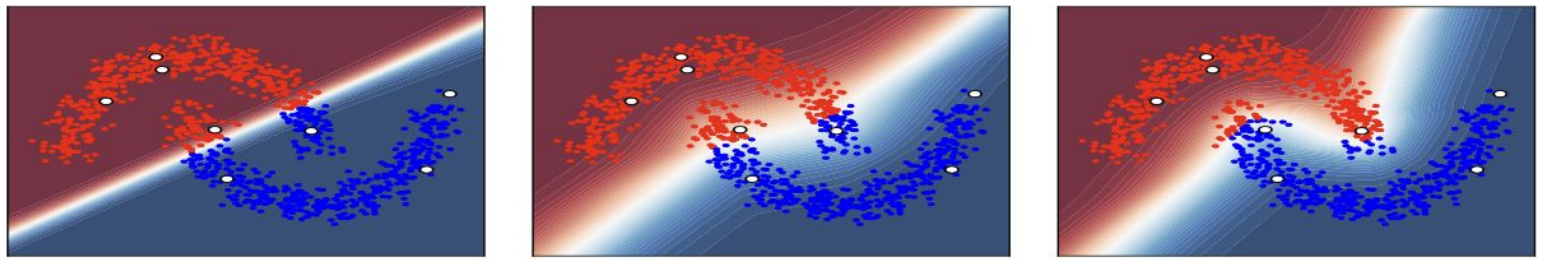


Fig. 1. Pseudo-labeling in the "two moons" data (4 labels/class) for 1000 samples. From left to right: no mixup, mixup, and mixup with a minimum number of labeled samples per mini-batch. We use an NN classifier with one hidden layer with 50 hidden units as in [20]. Best viewed in color.

Arazo, et. al., IJCNN 2020, Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning

Reducing Confirmation Bias

TABLE V

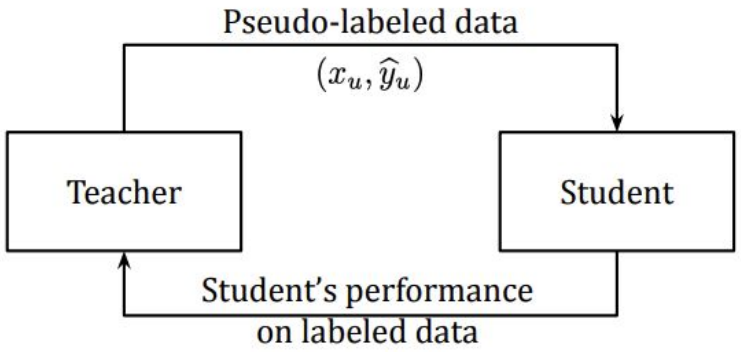
TEST ERROR IN CIFAR-10/100 FOR THE PROPOSED APPROACH USING THE 13-CNN NETWORK. (*) DENOTES THAT WE HAVE RUN THE ALGORITHM. BOLD INDICATES LOWEST ERROR. WE REPORT AVERAGE AND STANDARD DEVIATION OF 3 RUNS WITH DIFFERENT LABELED/UNLABELED SPLITS.

	CIFAR-10			CIFAR-100	
Labeled images	500	1000	4000	4000	10000
Supervised (C)*	43.64 ± 1.21	34.83 ± 1.15	19.26 ± 0.26	54.49 ± 0.53	41.14 ± 0.26
Supervised (M)*	37.60 ± 0.65	28.59 ± 1.21	15.94 ± 0.26	52.70 ± 0.28	39.42 ± 0.37
Consistency regularization methods					
Π model	-	-	12.36 ± 0.31	-	39.19 ± 0.36
TE	-	-	12.16 ± 0.24	-	38.65 ± 0.51
MT	27.45 ± 2.64	19.04 ± 0.51	11.41 ± 0.25	45.36 ± 0.49	36.08 ± 0.51
Π model-SN	-	21.23 ± 1.27	11.00 ± 0.13	-	37.97 ± 0.29
MA-DNN	-	-	11.91 ± 0.22	-	34.51 ± 0.61
Deep-Co	-	-	9.03 ± 0.18	-	38.77 ± 0.28
MT-TSSDL	-	18.41 ± 0.92	9.30 ± 0.55	-	-
MT-LP	24.02 ± 2.44	16.93 ± 0.70	10.61 ± 0.28	43.73 ± 0.20	35.92 ± 0.47
MT-CCL	-	16.99 ± 0.71	10.63 ± 0.22	-	34.81 ± 0.52
MT-fast-SWA	-	15.58 ± 0.12	9.05 ± 0.21	-	34.10 ± 0.31
ICT	-	15.48 ± 0.78	7.29 ± 0.02	-	-
Pseudo-labeling methods					
TSSDL	-	21.13 ± 1.17	10.90 ± 0.23	-	-
LP	32.40 ± 1.80	22.02 ± 0.88	12.69 ± 0.29	46.20 ± 0.76	38.43 ± 1.88
Ours*	**8.80 ± 0.45**	**6.85 ± 0.15**	**5.97 ± 0.15**	**37.55 ± 1.09**	**32.15 ± 0.50**

Arazo, et. al., IJCNN 2020, Pseudo-Labeling and Confirmation Bias in Deep Semi-Supervised Learning

Meta Pseudo Labels

- Motivation
  - Conventional self-training assumes the teacher generates good pseudo labels.
  - However, an **imperfect teacher** can produce incorrect supervision.
- Key Idea
  - Instead of fixing the teacher,
  - learn the teacher to **generate better pseudo labels**.
  - The teacher is optimized based on **how much the student improves**.



Pham, et. al., 2000, Meta Pseudo Labels

Meta Pseudo Labels

- Instead of directly evaluating the Teacher,
  - the Teacher is updated based on how well the Student performs after learning with the pseudo-labels created by the Teacher.
- Step 1. Student Update (Inner Optimization)
  - Train the student using pseudo labels generated by the teacher

$$\theta_S^{\mathrm{PL}}(\theta_T) = \arg\min_{\theta_S} \mathcal{L}_u(\theta_T, \theta_S) = \arg\min_{\theta_S} \mathbb{E}_{\mathbf{u}\sim\mathcal{U}} \mathrm{CE}[(f_{\theta_T}(\mathbf{u}), f_{\theta_S}(\mathbf{u}))]$$

- Step 2. Teacher Update (Outer Optimization)
  - Update the teacher so that the resulting student achieves lower supervised loss on labeled data
  - Evaluation is performed using correct answers (labels)

$$\min_{\theta_T} \mathcal{L}_s(\theta_S^{\mathrm{PL}}(\theta_T)) = \min_{\theta_T} \mathbb{E}_{(\mathbf{x}^l, y)\in\mathcal{X}} \mathrm{CE}[y, f_{\theta_S}(\mathbf{x}^l)]$$

Pham, et. al., 2000, Meta Pseudo Labels

Meta Pseudo Labels

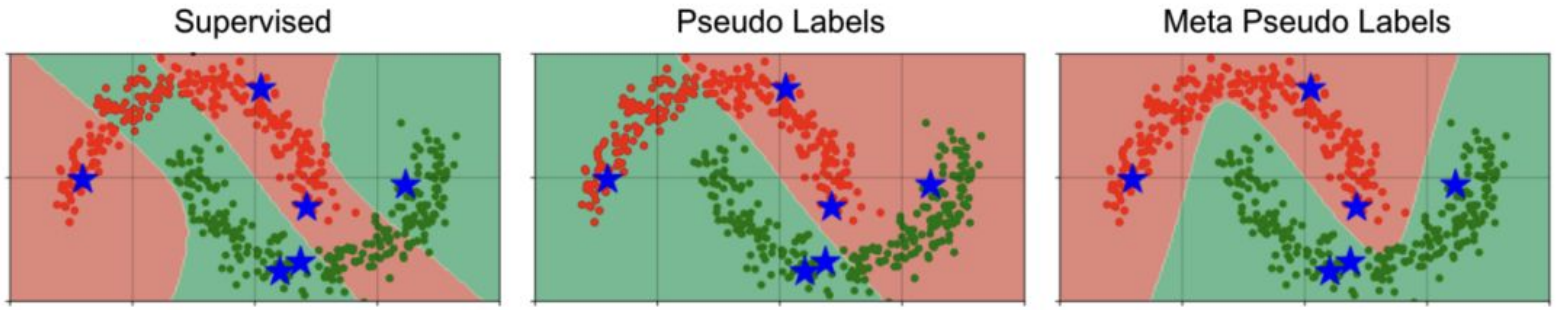


Figure 2: An illustration of the importance of feedback in Meta Pseudo Labels (right). In this example, Meta Pseudo Labels works better than Supervised Learning (left) and Pseudo Labels (middle) on the simple TwoMoon dataset. More details are in Section 3.1.

Pham, et. al., 2000, Meta Pseudo Labels

Meta Pseudo Labels

	Method	CIFAR-10-4K (mean ± std)	SVHN-1K (mean ± std)	ImageNet-10% Top-1	ImageNet-10% Top-5
Label Propagation Methods	Temporal Ensemble [35]	83.63 ± 0.63	92.81 ± 0.27	–	
	Mean Teacher [64]	84.13 ± 0.28	94.35 ± 0.47	–	
	VAT + EntMin [44]	86.87 ± 0.39	94.65 ± 0.19	–	83.39
	LGA + VAT [30]	87.94 ± 0.19	93.42 ± 0.36	–	
	ICT [71]	92.71 ± 0.02	96.11 ± 0.04	–	
	MixMatch [5]	93.76 ± 0.06	96.73 ± 0.31	–	
	ReMixMatch [4]	94.86 ± 0.04	97.17 ± 0.30	–	
	EnAET [72]	94.65	97.08	–	
	FixMatch [58]	95.74 ± 0.05	97.72 ± 0.38	71.5	89.1
	UDA* [76]	94.53 ± 0.18	97.11 ± 0.17	68.07	88.19
Self-Supervised Methods	SimCLR [8, 9]	–	–	71.7	90.4
	MOCOv2 [10]	–	–	71.1	–
	PCL [38]	–	–	–	85.6
	PIRL [43]	–	–	–	84.9
	BYOL [21]	–	–	68.8	89.0
	Meta Pseudo Labels	**96.11 ± 0.07**	**98.01 ± 0.07**	**73.89**	**91.38**
	Supervised Learning with full dataset*	94.92 ± 0.17	97.41 ± 0.16	76.89	93.27

Table 2: Image classification accuracy on CIFAR-10-4K, SVHN-1K, and ImageNet-10%. Higher is better. For CIFAR-10-4K and SVHN-1K, we report mean ± std over 10 runs, while for ImageNet-10%, we report Top-1/Top-5 accuracy of a single run. For fair comparison, we only include results that share the same model architecture: WideResNet-28-2 for CIFAR-10-4K and SVHN-1K, and ResNet-50 for ImageNet-10%. * indicates our implementation which uses the same experimental protocols. Except for UDA, results in the first two blocks are from representative important papers, and hence do not share the same controlled environment with ours.

Pham, et. al., 2000, Meta Pseudo Labels

Pseudo Labeling with Consistency Regularization

Pseudo Labeling with Consistency Regularization

- It is possible to **combine** the above two approaches together, running semi-supervised learning with both **pseudo labeling and consistency training.**

MixMatch

- Key Idea
  - Rather than proposing a new component,
  - MixMatch unifies several successful semi-supervised learning techniques into a single framework.
- Main Components
  - 1. Consistency Regularization
    - Encourage consistent predictions under perturbations.
  - 2. Entropy Minimization
    - Encourage confident predictions on unlabeled data.
  - 4. Pseudo Labeling
    - Generate targets for unlabeled samples.
  - 4. Label Sharpening
    - Convert soft predictions into confident targets.
  - 5. MixUp
    - Interpolate labeled and unlabeled samples.

MixMatch is the first holistic framework that combines multiple SSL ideas into a unified training pipeline.

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

MixMatch

Given a batch of labeled data $\mathcal{X}$ and unlabeled data $\mathcal{U}$, we create augmented versions of them via $\text{MixMatch}(.)$, $\bar{\mathcal{X}}$ and $\bar{\mathcal{U}}$, containing augmented samples and guessed labels for unlabeled examples.

$$\bar{\mathcal{X}}, \bar{\mathcal{U}} = \text{MixMatch}(\mathcal{X}, \mathcal{U}, T, K, \alpha)$$

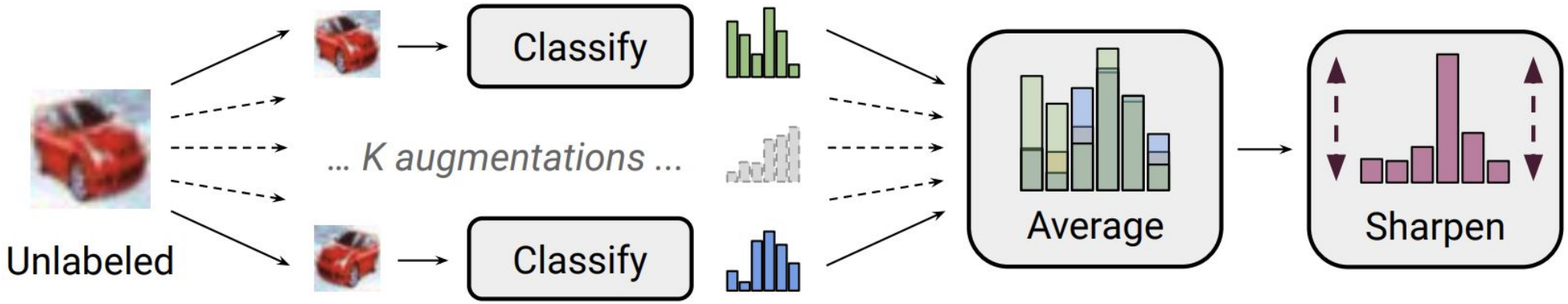
MixUp

$$\lambda \sim \text{Beta}(\alpha, \alpha)$$
$$\lambda' = \max(\lambda, 1-\lambda)$$
$$x' = \lambda' x_1 + (1-\lambda') x_2$$
$$p' = \lambda' p_1 + (1-\lambda') p_2$$

$$\hat{\mathcal{X}} = \left((\hat{x}_b, p_b); b \in (1, \ldots, B)\right)$$
$$\hat{\mathcal{U}} = \left((\hat{u}_{b,k}, q_b); b \in (1, \ldots, B), k \in (1, \ldots, K)\right)$$

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

MixMatch



Label Guessing for Unlabeled Data

$$\bar{q}_b = \frac{1}{K} \sum_{k=1}^{K} \mathrm{p}_{\mathrm{model}}(y \mid \hat{u}_{b,k}; \theta)$$

Sharpening

$$\mathrm{Sharpen}(p, T)_i := p_i^{\frac{1}{T}} \Big/ \sum_{j=1}^{L} p_j^{\frac{1}{T}}$$

adjusting the “temperature” of this categorical distribution

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

MixMatch

- Losses for both label and unlabeled data.

$$\mathcal{L}_s^{\mathrm{MM}} = \frac{1}{|\bar{\mathcal{X}}|} \sum_{(\bar{\mathbf{x}}^l, y) \in \bar{\mathcal{X}}} D[y, p_\theta(y \mid \bar{\mathbf{x}}^l)]$$

$$\mathcal{L}_u^{\mathrm{MM}} = \frac{1}{L|\bar{\mathcal{U}}|} \sum_{(\bar{\mathbf{u}}, \hat{y}) \in \bar{\mathcal{U}}} \|\hat{y} - p_\theta(y \mid \bar{\mathbf{u}})\|_2^2$$

where T is the sharpening temperature to reduce the guessed label overlap; K is the number of augmentations generated per unlabeled example; α is the parameter in MixUp.

MixMatch combines several successful SSL techniques into a unified training framework.

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

MixMatch

Method	CIFAR-10	CIFAR-100
Mean Teacher [44]	6.28	-
SWA [2]	5.00	28.80
MixMatch	4.95 ± 0.08	25.88 ± 0.30

Table 1: CIFAR-10 and CIFAR-100 error rate (with 4,000 and 10,000 labels respectively) with larger models (26 million parameters).

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

MixMatch

Ablation	250 labels	4000 labels
MixMatch	11.80	6.00
MixMatch without distribution averaging ($K = 1$)	17.09	8.06
MixMatch with $K = 3$	11.55	6.23
MixMatch with $K = 4$	12.45	5.88
MixMatch without temperature sharpening ($T = 1$)	27.83	10.59
MixMatch with parameter EMA	11.86	6.47
MixMatch without MixUp	39.11	10.97
MixMatch with MixUp on labeled only	32.16	9.22
MixMatch with MixUp on unlabeled only	12.35	6.83
MixMatch with MixUp on separate labeled and unlabeled	12.26	6.50
Interpolation Consistency Training [45]	38.60	6.81

Table 4: Ablation study results. All values are error rates on CIFAR-10 with 250 or 4000 labels.

Berthelot, et. al., NeurIPS 2019, MixMatch: A Holistic Approach to Semi-Supervised Learning

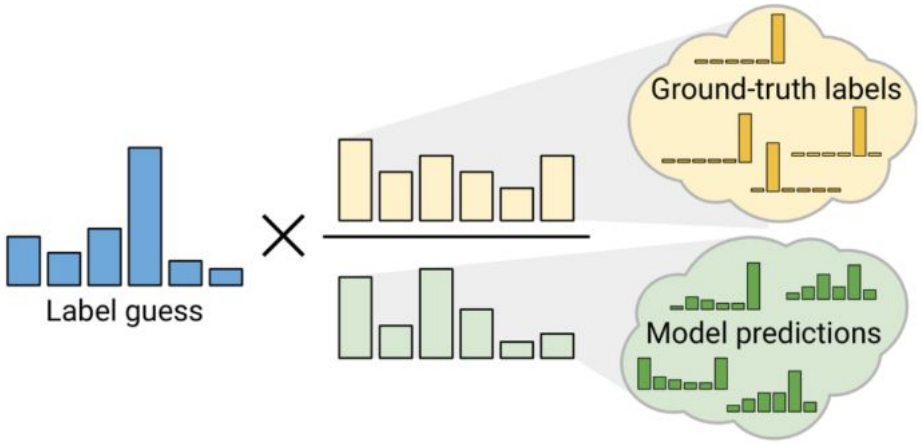
ReMixMatch

- Motivation
  - Although MixMatch achieved strong performance, it still suffers from two limitations:
    - 1. Class imbalance in pseudo labels.
      - → Distribution Alignment
        - Align pseudo-label distribution with the expected class distribution.
    - 2. Unstable pseudo labels under strong augmentations.
      - → Augmentation Anchoring
        - Generate pseudo labels from weak augmentations and enforce consistency on strong augmentations.
    - 3. Additional Regularization
      - Rotation prediction (self-supervised learning)

Berthelot, et. al., 2020, ReMixMatch: Semi-Supervised Learning with Distribution Alignment and Augmentation Anchoring

ReMixMatch

- Pseudo labels may not follow the true class distribution.



Distribution alignment.

$$\text{Normalize}\left(\frac{p_\theta(y|\mathbf{u})p(y)}{\tilde{p}(\hat{y})}\right)$$

The model suppresses over-represented classes and boosts under-represented classes.
→ **Decrease the frequency of classes that appear too often, and increase the frequency of classes that appear too rarely.**

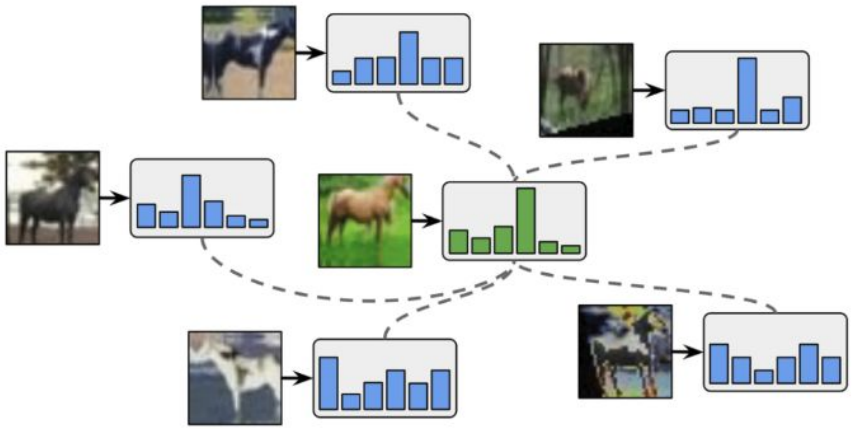
Blue histogram : prediction for one unlabeled sample (label guess)
Green histogram : estimated class distribution of all pseudo labels
Yellow histogram : class distribution of the labeled dataset (target prior)

Distribution Alignment rescales the blue prediction using the ratio between the yellow and green distributions.

Berthelot, et. al., 2020, ReMixMatch: Semi-Supervised Learning with Distribution Alignment and Augmentation Anchoring

ReMixMatch

- Motivation
  - Strong augmentations improve robustness, but they often produce unreliable pseudo labels.
- Key Idea
  - Generate pseudo labels from a weakly augmented image (anchor),
  - and enforce consistency on strongly augmented versions.



Augmentation anchoring.

🟩 Green histogram

Prediction from the weakly augmented anchor image
Used as the target pseudo label

🟦 Blue histograms

Predictions from multiple strongly augmented images
Trained to match the anchor prediction

The pseudo label is generated **once** from the anchor image and shared across all strong augmentations.

Berthelot, et. al., 2020, ReMixMatch: Semi-Supervised Learning with Distribution Alignment and Augmentation Anchoring

ReMixMatch

- Why does it help?
  - Weak augmentations provide stable pseudo labels.
  - Strong augmentations improve augmentation invariance.
  - Decouples target generation from consistency training.

Use a reliable weakly augmented image as an anchor to supervise multiple strongly augmented views.

ReMixMatch

MixMatch	ReMixMatch
Label Guessing	✓
Sharpening	✓
MixUp	✓
Consistency Loss	✓
Distribution Alignment	**✓ (New)**
Augmentation Anchoring	**✓ (New)**
Self-Supervised Rotation Loss	**✓ (New)**

Berthelot, et. al., 2020, ReMixMatch: Semi-Supervised Learning with Distribution Alignment and Augmentation Anchoring

ReMixMatch

	CIFAR-10			SVHN		
Method	250 labels	1000 labels	4000 labels	250 labels	1000 labels	4000 labels
VAT	36.03±2.82	18.64±0.40	11.05±0.31	8.41±1.01	5.98±0.21	4.20±0.15
Mean Teacher	47.32±4.71	17.32±4.00	10.36±0.25	6.45±2.43	3.75±0.10	3.39±0.11
MixMatch	11.08±0.87	7.75±0.32	6.24±0.06	3.78±0.26	3.27±0.31	2.89±0.06
ReMixMatch	6.27±0.34	5.73±0.16	5.14±0.04	3.10±0.50	2.83±0.30	2.42±0.09
UDA, reported*	8.76±0.90	5.87±0.13	5.29±0.25	2.76±0.17	2.55±0.09	2.47±0.15

Table 1: Results on CIFAR-10 and SVHN. * For UDA, due to adaptation difficulties, we report the results from Xie et al. (2019) which are not comparable to our results due to a different network implementation, training procedure, etc. For VAT, Mean Teacher, and MixMatch, we report results using our reimplementation, which makes them directly comparable to ReMixMatch's scores.

Berthelot, et. al., 2020, ReMixMatch: Semi-Supervised Learning with Distribution Alignment and Augmentation Anchoring

FixMatch

- Previous SSL Methods
  - MixMatch and ReMixMatch combine multiple components.
    - Label Guessing
    - Sharpening
    - MixUp
    - Distribution Alignment
    - Multiple Augmentations

They achieve strong performance, but the training pipeline becomes increasingly complicated.

Can we achieve similar or even better performance with a much simpler framework?

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

- Instead of combining many techniques, FixMatch relies on only three simple ideas.
  - ① Weak Augmentation
    - Generate a reliable pseudo label
  - ② Confidence Threshold
    - Discard uncertain predictions
  - ③ Strong Augmentation
    - Train the model to predict the same label

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

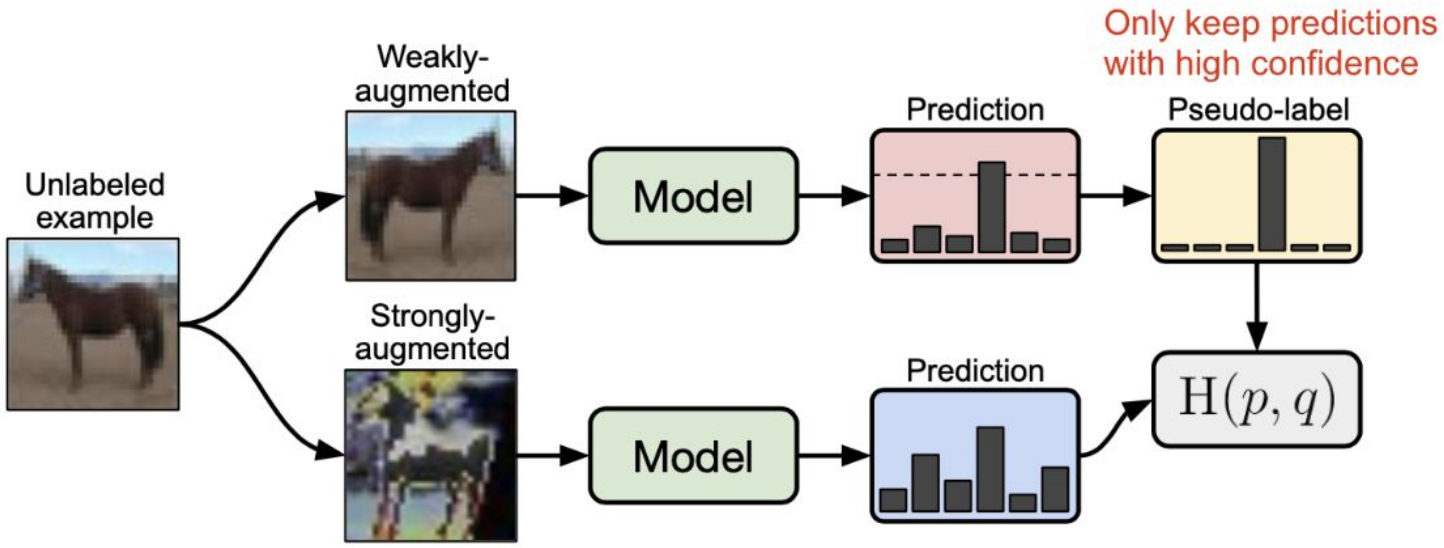
- Weak augmentation
  - produces a stable prediction
  - easier classification
- Strong augmentation
  - much harder input
  - improves robustness

Predict on an easy example, learn from a difficult one.

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

- A simpler framework using weak→strong consistency and confidence filtering.
  - High confidence leads to high-quality pseudo labels.

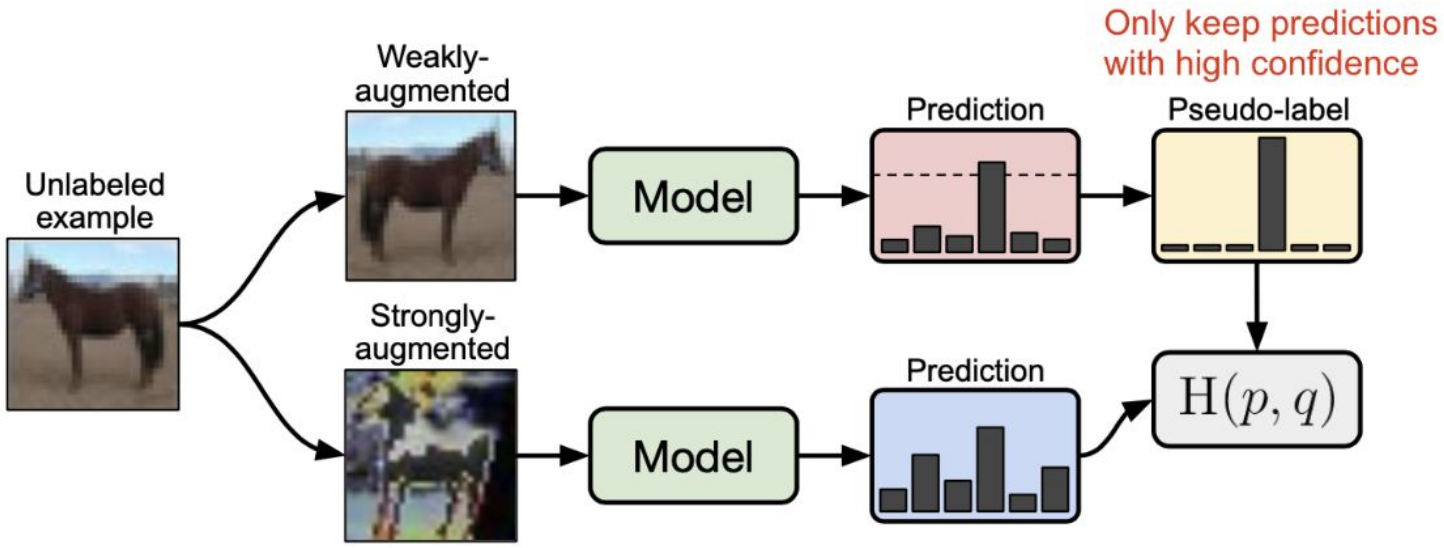


- Weak augmentation $\mathcal{A}_{\text{weak}}(.)$: A standard flip-and-shift augmentation
- Strong augmentation $\mathcal{A}_{\text{strong}}(.)$: AutoAugment, Cutout, RandAugment, CTAugment

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

- A simpler framework using weak→strong consistency and confidence filtering.
  - High confidence leads to high-quality pseudo labels.



$$\mathcal{L}_s = \frac{1}{B}\sum_{b=1}^{B} \mathrm{CE}[y_b, p_\theta(y \mid \mathcal{A}_{\text{weak}}(\mathbf{x}_b))]$$

$$\mathcal{L}_u = \frac{1}{\mu B}\sum_{b=1}^{\mu B} \mathbb{1}[\max(\hat{y}_b) \geq \tau]\ \mathrm{CE}(\hat{y}_b, p_\theta(y \mid \mathcal{A}_{\text{strong}}(\mathbf{u}_b)))$$

- Weak augmentation $\mathcal{A}_{\text{weak}}(.)$: A standard flip-and-shift augmentation
- Strong augmentation $\mathcal{A}_{\text{strong}}(.)$: AutoAugment, Cutout, RandAugment, CTAugment

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

	CIFAR-10			CIFAR-100			SVHN			STL-10
Method	40 labels	250 labels	4000 labels	400 labels	2500 labels	10000 labels	40 labels	250 labels	1000 labels	1000 labels
Π-Model	-	54.26±3.97	14.01±0.38	-	57.25±0.48	37.88±0.11	-	18.96±1.92	7.54±0.36	26.23±0.82
Pseudo-Labeling	-	49.78±0.43	16.09±0.28	-	57.38±0.46	36.21±0.19	-	20.21±1.09	9.94±0.61	27.99±0.83
Mean Teacher	-	32.32±2.30	9.19±0.19	-	53.91±0.57	35.83±0.24	-	3.57±0.11	3.42±0.07	21.43±2.39
MixMatch	47.54±11.50	11.05±0.86	6.42±0.10	67.61±1.32	39.94±0.37	28.31±0.33	42.55±14.53	3.98±0.23	3.50±0.28	10.41±0.61
UDA	29.05±5.93	8.82±1.08	4.88±0.18	59.28±0.88	33.13±0.22	24.50±0.25	52.63±20.51	5.69±2.76	**2.46**±0.24	7.66±0.56
ReMixMatch	**19.10**±9.64	**5.44**±0.05	4.72±0.13	**44.28**±2.06	**27.43**±0.31	**23.03**±0.56	**3.34**±0.20	**2.92**±0.48	2.65±0.08	**5.23**±0.45
FixMatch (RA)	**13.81**±3.37	**5.07**±0.65	**4.26**±0.05	48.85±1.75	28.29±0.11	**22.60**±0.12	**3.96**±2.17	**2.48**±0.38	**2.28**±0.11	7.98±1.50
FixMatch (CTA)	**11.39**±3.35	**5.07**±0.33	**4.31**±0.15	49.95±3.01	28.64±0.24	23.18±0.11	7.65±7.65	**2.64**±0.64	**2.36**±0.19	**5.17**±0.63

RA: RandAugment, CTA: CTAugment

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

FixMatch

- Why Does FixMatch Work?
- According to the ablation studies of FixMatch,
  - **Confidence** thresholding is more effective than temperature sharpening.
    - Once low-confidence predictions are filtered out, sharpening provides little additional benefit.
  - **Weak augmentation** should be used for pseudo-label generation.
    - Strong augmentation introduces excessive perturbations, leading to unreliable pseudo labels.
  - **Strong augmentation** is essential for consistency learning.
    - Training with strongly augmented images significantly improves robustness and generalization.
  - The combination of weak→strong augmentation and confidence filtering is the key to FixMatch.

Sohn, et. al., NeurIPS 2021, FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

DivideMix

- Existing SSL methods assume that
  - labeled samples are clean
  - only unlabeled data are unreliable
- But in Practice... Labeled datasets may also contain incorrect labels.

Can semi-supervised learning be used to handle noisy labels?
→ Instead of trusting all labeled samples, identify unreliable labels and treat them as unlabeled data.

Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

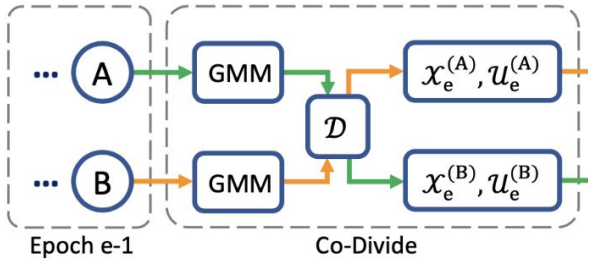
- Key Idea
  - Divide the labeled dataset into
    - Clean samples
    - Noisy samples
  - using the training loss.
- Discard unreliable labels and treat those samples as unlabeled data.
  - Then optimize the model using MixMatch.

- MixMatch is a strong SSL framework that effectively utilizes unlabeled data.
- DivideMix reformulates noisy-label learning as a semi-supervised learning problem.

Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

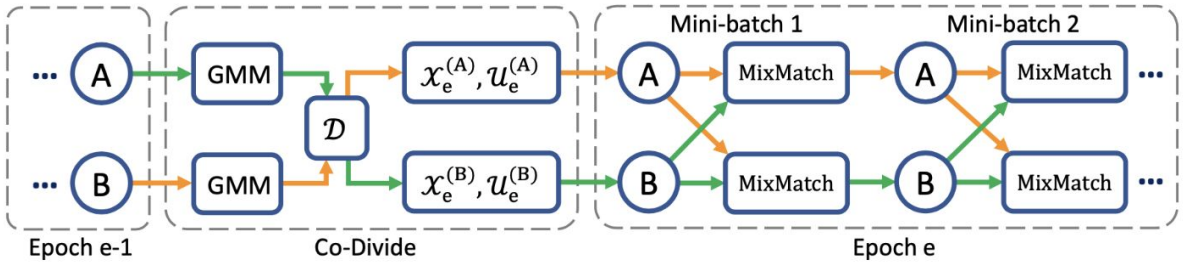
- (1) Train Two Networks
  - Network A, Network B
  - To reduce confirmation bias by judging clean/noisy from different perspectives.
- (2) Co-Divide
  - A ⟶ GMM, B ⟶ GMM
  - Each network calculates the sample loss at the current epoch.
  - By fitting a 2-component GMM to the loss distribution,
    - Low-loss → Clean
    - High-loss → Noisy
  - It separates into.
    - Instead of using its own judgment, it relies on the "clean" or "noisy" classification determined by the other party.



Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

- (3) Construct Training Sets
  - It automatically generates the labeled and unlabeled datasets required for MixMatch.
- (4) MixMatch Training
  - DivideMix does not create a new learning algorithm;
  - rather, it is a method for generating the labeled and unlabeled sets used by MixMatch.
- (5) Repeat



Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

- Q1. Why use GMM?
  - During the early stages of training:
    - Clean samples show a rapid decrease in loss.
    - Noisy samples retain a high loss.
  - This is because the loss histogram
    - roughly appears as two clusters:
- Q2. Are noisy samples discarded?
  - No.
  - Only the ground-truth labels are discarded.
  - In other words, they are treated as unlabeled data.

Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

Algorithm 1: DivideMix. Line 4-8: co-divide; Line 17-18: label co-refinement; Line 20: label co-guessing.

Input: $\theta^{(1)}$ and $\theta^{(2)}$, training dataset $(\mathcal{X}, \mathcal{Y})$, clean probability threshold τ, number of augmentations M, sharpening temperature T, unsupervised loss weight λ_u, Beta distribution parameter α for MixMatch.

$\theta^{(1)}, \theta^{(2)} = \text{WarmUp}(\mathcal{X}, \mathcal{Y}, \theta^{(1)}, \theta^{(2)})$ *// standard training (with confidence penalty)*

while $e < \text{MaxEpoch}$ **do** — Co-divide

  $\mathcal{W}^{(2)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(1)})$ *// model per-sample loss with $\theta^{(1)}$ to obtain clean proabability for $\theta^{(2)}$*

  $\mathcal{W}^{(1)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(2)})$ *// model per-sample loss with $\theta^{(2)}$ to obtain clean proabability for $\theta^{(1)}$*

  for $k = 1, 2$ **do** *// train the two networks one by one*

    $\mathcal{X}_e^{(k)} = \{(x_i, y_i, w_i) | w_i \geq \tau, \forall (x_i, y_i, w_i) \in (\mathcal{X}, \mathcal{Y}, \mathcal{W}^{(k)})\}$ *// labeled training set for $\theta^{(k)}$*

    $\mathcal{U}_e^{(k)} = \{x_i | w_i < \tau, \forall (x_i, w_i) \in (\mathcal{X}, \mathcal{W}^{(k)})\}$ *// unlabeled training set for $\theta^{(k)}$*

    for iter $= 1$ **to** num_iters **do**

      From $\mathcal{X}_e^{(k)}$, draw a mini-batch $\{(x_b, y_b, w_b); b \in (1, ..., B)\}$

      From $\mathcal{U}_e^{(k)}$, draw a mini-batch $\{u_b; b \in (1, ..., B)\}$

      for $b = 1$ **to** B **do**

        for $m = 1$ **to** M **do**

          $\hat{x}_{b,m} = \text{Augment}(x_b)$ *// apply m^{th} round of augmentation to x_b*

          $\hat{u}_{b,m} = \text{Augment}(u_b)$ *// apply m^{th} round of augmentation to u_b*

        end

        Label co-refine

        $p_b = \frac{1}{M} \sum_m \text{p}_{\text{model}}(\hat{x}_{b,m}; \theta^{(k)})$ *// average the predictions across augmentations of x_b*

        $\bar{y}_b = w_b y_b + (1 - w_b) p_b$

        // refine ground-truth label guided by the clean probability produced by the other network

        $\hat{y}_b = \text{Sharpen}(\bar{y}_b, T)$ *// apply temperature sharpening to the refined label*

        Label co-guessing

        $\bar{q}_b = \frac{1}{2M} \sum_m \left(\text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(1)}) + \text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(2)})\right)$

        // co-guessing: average the predictions from both networks across augmentations of u_b

        $q_b = \text{Sharpen}(\bar{q}_b, T)$ *// apply temperature sharpening to the guessed label*

      end

      $\hat{\mathcal{X}} = \{(\hat{x}_{b,m}, \hat{y}_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented labeled mini-batch*

      $\hat{\mathcal{U}} = \{(\hat{u}_{b,m}, q_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented unlabeled mini-batch*

      $\mathcal{L}_\mathcal{X}, \mathcal{L}_\mathcal{U} = \text{MixMatch}(\hat{\mathcal{X}}, \hat{\mathcal{U}})$ *// apply MixMatch*

      $\mathcal{L} = \mathcal{L}_\mathcal{X} + \lambda_u \mathcal{L}_\mathcal{U} + \lambda_r \mathcal{L}_{\text{reg}}$ *// total loss*

      $\theta^{(k)} = \text{SGD}(\mathcal{L}, \theta^{(k)})$ *// update model parameters*

    end

  end

end

Label Co-Refinement: Refines labels for samples classified as "Clean" by combining the ground-truth (GT) label with the model's prediction.

Label Co-Guessing: Generates pseudo-labels for noisy (unlabeled) samples by averaging the predictions from the two networks.

Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

DivideMix

Dataset		CIFAR-10					CIFAR-100			
Noise type		Sym.				Asym.	Sym.			
Methods/Noise ratio		20%	50%	80%	90%	40%	20%	50%	80%	90%
DivideMix	Best	**96.1**	**94.6**	**93.2**	**76.0**	**93.4**	**77.3**	**74.6**	**60.2**	**31.5**
	Last	**95.7**	**94.4**	**92.9**	**75.4**	**92.1**	**76.9**	**74.2**	**59.6**	**31.0**
DivideMix with $\theta^{(1)}$ test	Best	95.2	94.2	93.0	75.5	92.7	75.2	72.8	58.3	29.9
	Last	95.0	93.7	92.4	74.2	91.4	74.8	72.1	57.6	29.2
DivideMix w/o co-training	Best	95.0	94.0	92.6	74.3	91.9	74.8	72.3	56.7	27.7
	Last	94.8	93.3	92.2	73.2	90.6	74.1	71.7	56.3	27.2
DivideMix w/o label refinement	Best	96.0	94.6	93.0	73.7	87.7	76.9	74.2	58.7	26.9
	Last	95.5	94.2	92.7	73.0	86.3	76.4	73.9	58.2	26.3
DivideMix w/o augmentation	Best	95.3	94.1	92.2	73.9	89.5	76.5	73.1	58.2	26.9
	Last	94.9	93.5	91.8	73.0	88.4	76.2	72.6	58.0	26.4
Divide and MixMatch	Best	94.1	92.8	89.7	70.1	86.5	73.7	70.5	55.3	25.0
	Last	93.5	92.3	89.1	68.6	85.2	72.4	69.7	53.9	23.7

Table 5: Ablation study results in terms of test accuracy (%) on CIFAR-10 and CIFAR-100.

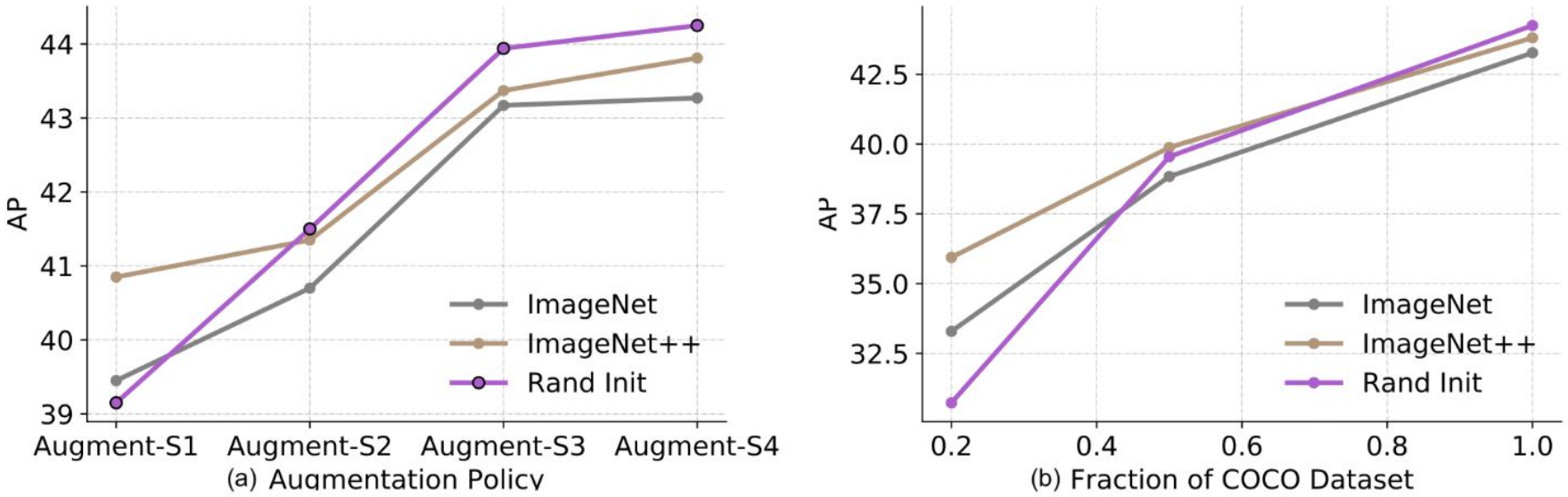
Li, et. al., ICML 2020, DivideMix: Learning with Noisy Labels as Semi-supervised Learning

Combined with Powerful Pre-Training

- Recent studies show that self-supervised pre-training and semi-supervised learning are highly **complementary**.
- Key Observations
  - Pre-training provides a strong feature representation.
  - Semi-supervised learning further adapts the model using limited labeled data.
  - Combining both often achieves better performance than either approach alone.
- Practical Implications
  - SSL is not a replacement for pre-training.
  - Modern vision systems often use both:
    - Self-supervised pre-training
    - Semi-supervised fine-tuning

Pre-training learns general representations, while semi-supervised learning adapts them to downstream tasks with limited annotations.

Combined with Powerful Pre-Training



(Left) Effect of Strong Data Augmentation

- As augmentation becomes stronger, the benefit of pre-training decreases.
- Strong augmentation can partially compensate for the lack of pre-training.

(Right) Effect of Labeled Data Size

- Pre-training is most beneficial when labeled data are scarce.
- As more labeled data become available, the performance gap gradually narrows.

Zoph, et. al., NeurIPS 2020, Rethinking Pre-training and Self-training

Combined with Powerful Pre-Training

- Key Findings
  - **Pre-training** is most beneficial when labeled data are **scarce**.
    - Its benefit gradually diminishes as more labeled data become available.
  - **Self-training** remains **effective** even with strong augmentation and abundant labeled data.
    - It can improve performance beyond pre-training alone.
  - Pre-training and self-training are **complementary**.
    - Combining both generally yields better performance than using either individually.
  - Task **mismatch** limits the effectiveness of pre-training.
    - Joint supervised and self-supervised training helps reduce this mismatch.
  - **Self-training** is computationally more expensive than conventional fine-tuning.

Zoph, et. al., NeurIPS 2020, Rethinking Pre-training and Self-training

Towards Modern Learning Paradigms

- Recent vision systems combine pre-training, fine-tuning, and self-training into a unified pipeline.
- Three-Step Procedure
  - 1. Self-Supervised Pre-training
    - Learn general visual representations from large-scale unlabeled data.
    - Build a strong task-agnostic backbone.
  - 2. Supervised Fine-tuning
    - Adapt the pretrained model using a small labeled dataset.
    - Larger models typically benefit more from pre-training.
  - 3. Self-Training
    - Generate pseudo labels for additional unlabeled data.
    - Further improve task-specific performance through knowledge distillation.
- Key Insight
  - **Pre-training learns general representations**, **fine-tuning adapts them to the target task**, and **self-training further exploits unlabeled data for additional gains**.

Chen, et. al., NeurIPS 2020, Big Self-Supervised Models are Strong Semi-Supervised Learners

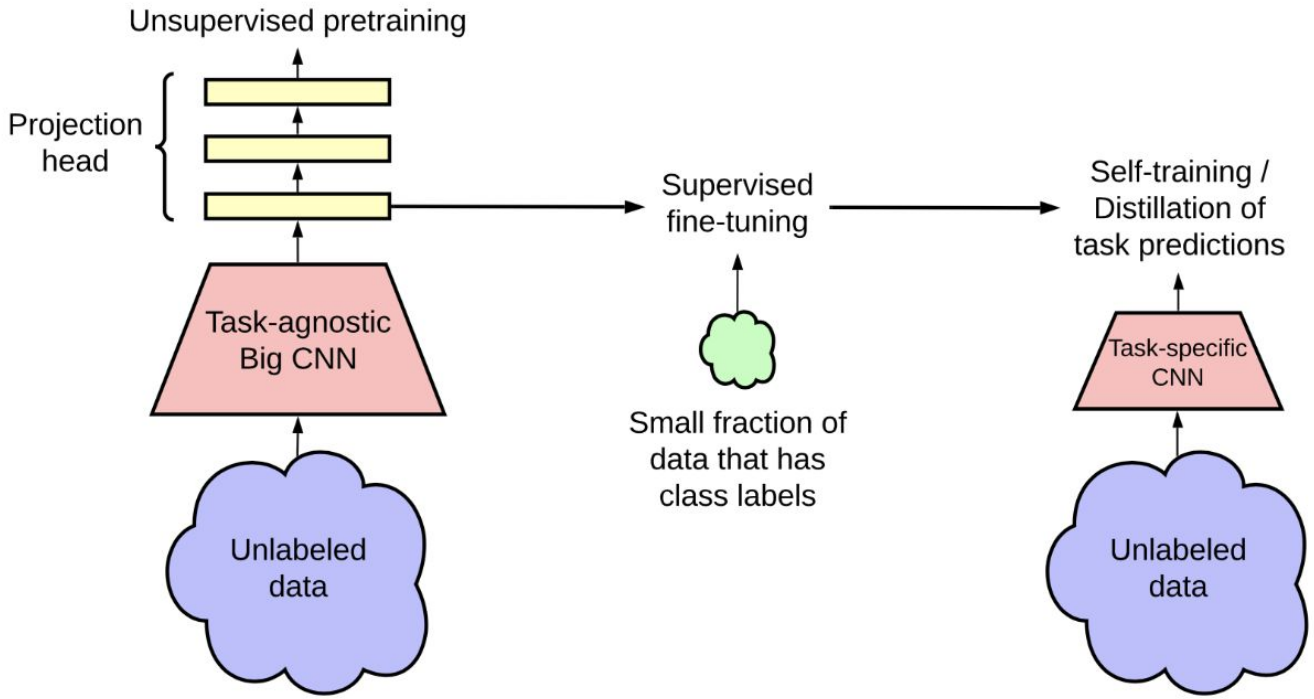
Towards Modern Learning Paradigms

- Recent vision systems combine pre-training, fine-tuning, and self-training into a unified pipeline.
- Three-Step Procedure
  - 1. Self-Supervised Pre-training
    - Learn general visual representations from large-scale unlabeled data.
    - Build a strong task-agnostic backbone.
  - 2. Supervised Fine-tuning
    - Adapt the pretrained model using a small labeled dataset.
    - Larger models typically benefit more from pre-training.
  - 3. Self-Training
    - Generate pseudo labels for additional unlabeled data.
    - Further improve task-specific performance through knowledge distillation.
- Key Insight
  - **Pre-training learns general representations**, **fine-tuning adapts them to the target task**, and **self-training further exploits unlabeled data for additional gains**.

Chen, et. al., NeurIPS 2020, Big Self-Supervised Models are Strong Semi-Supervised Learners

Towards Modern Learning Paradigms

$$\mathcal{L}_{\text{distill}} = -(1-\alpha) \underbrace{\sum_{(\mathbf{x}_i^l, y_i) \in \mathcal{X}} \left[\log p_{\theta_S}(y_i \mid \mathbf{x}_i^l)\right]}_{\text{Supervised loss}} - \alpha \underbrace{\sum_{\mathbf{u}_i \in \mathcal{U}} \left[\sum_{i=1}^{L} p_{\hat{\theta}_T}(y^{(i)} \mid \mathbf{u}_i; T) \log p_{\theta_S}(y^{(i)} \mid \mathbf{u}_i; T)\right]}_{\text{Distillation loss using unlabeled data}}$$



Chen, et. al., NeurIPS 2020, Big Self-Supervised Models are Strong Semi-Supervised Learners

Towards Modern Learning Paradigms

- Common design principles in modern SSL methods
  - Strong and diverse data augmentation (UDA, ReMixMatch, FixMatch)
  - Confidence-based pseudo-label filtering (FixMatch)
  - MixUp for regularization (MixMatch, Arazo et al.)
  - A minimum number of labeled samples per mini-batch (Arazo et al.)
  - Sharpening the pseudo-label distribution (MixMatch, ReMixMatch)

Towards Modern Learning Paradigms

- Today's state-of-the-art models rarely rely on a single learning paradigm.
- Instead, they integrate self-supervised pre-training, supervised fine-tuning, and semi-supervised self-training into a unified training pipeline.

Should the confidence threshold remain fixed throughout training?

Published as a conference paper at ICLR 2023

FREEMATCH: SELF-ADAPTIVE THRESHOLDING FOR SEMI-SUPERVISED LEARNING

Yidong Wang[1,2]*, **Hao Chen**[3]*, **Qiang Heng**[4], **Wenxin Hou**[5], **Yue Fan**[6],
Zhen Wu[7], **Jindong Wang**[1]†, **Marios Savvides**[3], **Takahiro Shinozaki**[2],
Bhiksha Raj[3], **Bernt Schiele**[6], **Xing Xie**[1]

[1]Microsoft Research Asia, [2]Tokyo Institute of Technology, [3]Carnegie Mellon University,
[4]North Carolina State University, [5]Microsoft STCA,
[6]Max-Planck-Institut für Informatik, [7]Nanjing University

Wang, et. al., ICML 2023, FreeMatch: Self-adaptive Thresholding for Semi-supervised Learning

Should the Confidence Threshold Remain Fixed?

- FixMatch uses a fixed confidence threshold.
  - The same threshold is used throughout training.
  - The same threshold is used for every class.
  - Low-confidence samples are excluded from training.

$$\mathcal{L}_s = \frac{1}{B}\sum_{b=1}^{B} \mathrm{CE}[y_b, p_\theta(y \mid \mathcal{A}_{\text{weak}}(\mathbf{x}_b))]$$

$$\mathcal{L}_u = \frac{1}{\mu B}\sum_{b=1}^{\mu B} \mathbb{1}[\max(\hat{y}_b) \geq \tau]\ \mathrm{CE}(\hat{y}_b, p_\theta(y \mid \mathcal{A}_{\text{strong}}(\mathbf{u}_b)))$$

where $\hat{y}_b$ is the pseudo label for an unlabeled example; μ is a hyperparameter that determines the relative sizes of $\mathcal{X}$ and $\mathcal{U}$.

- Weak augmentation $\mathcal{A}_{\text{weak}}(.)$: A standard flip-and-shift augmentation
- Strong augmentation $\mathcal{A}_{\text{strong}}(.)$: AutoAugment, Cutout, RandAugment, CTAugment

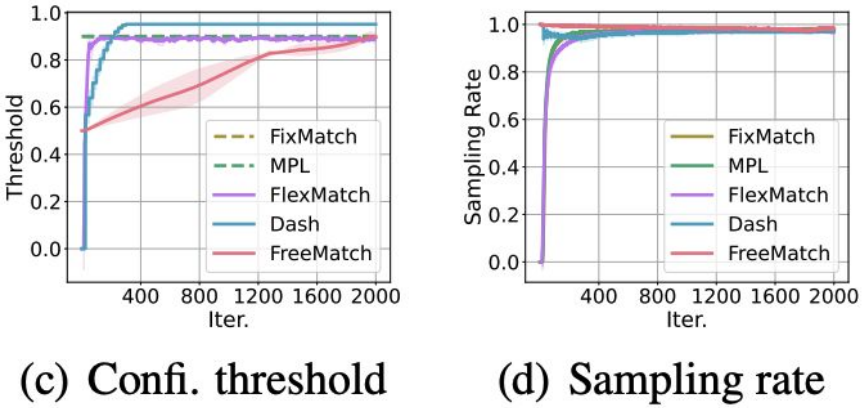
Should the confidence threshold remain fixed as the model improves?

The Problem with a Fixed Threshold

- Early training
  - Model confidence is still low.
  - A high threshold rejects most unlabeled samples.
  - The model receives little unsupervised supervision.
- Later training
  - Model confidence becomes higher.
  - A higher threshold helps filter incorrect pseudo-labels.
  - Different classes may require different thresholds.

FixMatch: Fixed at 0.95 throughout training

FreeMatch: Starts with a low threshold
- The threshold increases as the model's confidence rises



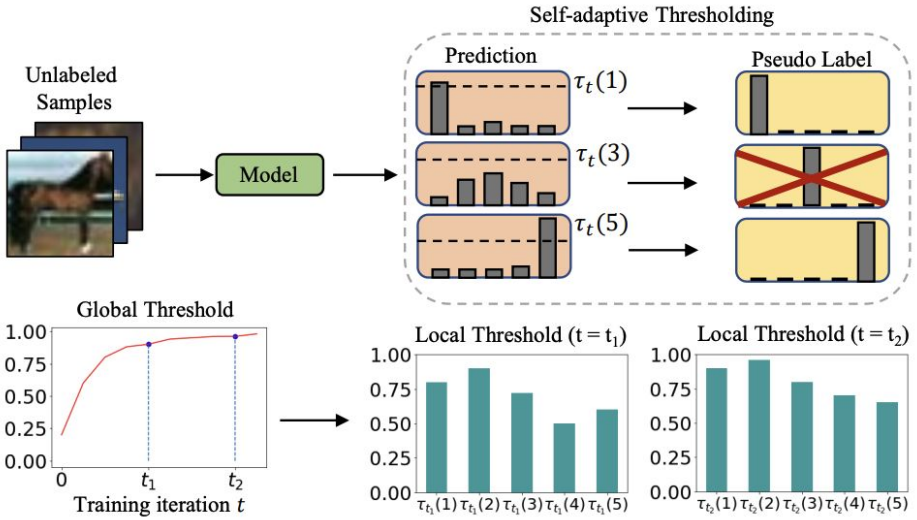
(c) Confi. threshold (d) Sampling rate

The desirable threshold depends on the model's learning status.

FreeMatch

FreeMatch contains two main components.

- Self-Adaptive Thresholding, SAT
  - Estimates the model's learning status
  - Adjusts the global threshold over time
  - Assigns a different threshold to each class
- Self-Adaptive Fairness, SAF
  - Prevents predictions from concentrating on a few classes
  - Encourages diverse class predictions



The model determines its thresholds from predictions on unlabeled data.

Self-Adaptive Global Threshold

- Compute the average confidence of unlabeled samples.
- Accumulate it with an exponential moving average.
- Use the result as the global threshold.
- The threshold increases as the model becomes more confident.

$$\tau_t = \begin{cases} \frac{1}{C}, & \text{if } t = 0, \\ \lambda \tau_{t-1} + (1-\lambda)\frac{1}{\mu B}\sum_{b=1}^{\mu B} \max(q_b), & \text{otherwise,} \end{cases}$$

- C: number of classes
- t: training iteration
- q_b: prediction on a weakly augmented unlabeled image
- $\max(q_b)$: confidence of sample b
- μB: number of unlabeled samples in the minibatch
- λ: EMA momentum
- τ_t: global threshold at iteration t

The global threshold follows the model's overall confidence.

Self-Adaptive Class-Specific Thresholds

- Easy classes receive relatively high thresholds.
- Difficult classes receive relatively low thresholds.
- Every class shares the same global learning trend.
- Local scaling accounts for different learning speeds.

$$\tilde{p}_t(c) = \begin{cases} \frac{1}{C}, & \text{if } t = 0, \\ \lambda \tilde{p}_{t-1}(c) + (1-\lambda)\frac{1}{\mu B}\sum_{b=1}^{\mu B} q_b(c), & \text{otherwise,} \end{cases}$$

$$\tau_t(c) = \text{MaxNorm}(\tilde{p}_t(c)) \cdot \tau_t = \frac{\tilde{p}_t(c)}{\max\{\tilde{p}_t(c) : c \in [C]\}} \cdot \tau_t,$$

One global threshold is modulated by class-specific learning progress.

Pseudo-Label Selection in FreeMatch

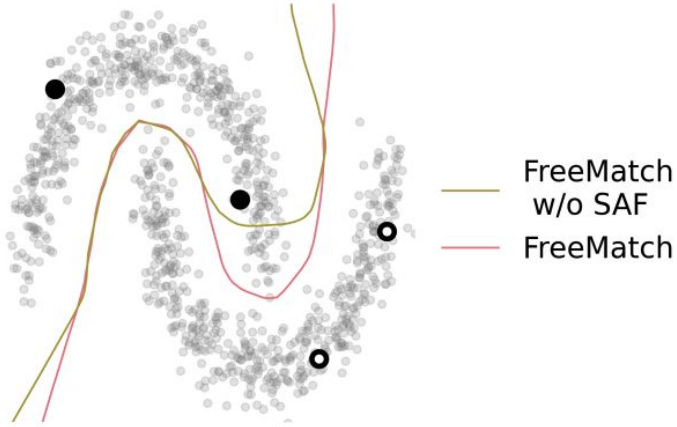
$$\mathcal{L}_u = \frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}(\max(q_b) > \tau_t(\arg\max(q_b)) \cdot \mathcal{H}(\hat{q}_b, Q_b).$$

FixMatch	FreeMatch
Fixed tau	Adaptive tau_t(c)
Same for all iterations	Changes during training
Same for all classes	Different for each class
Manually selected	Estimated from unlabeled predictions

Each sample is evaluated using the threshold of its predicted class.

Why Is Class Fairness Needed?

- With very few labeled samples:
  - Early predictions can favor a small number of classes.
  - Dominant classes receive more pseudo-labels.
  - More pseudo-labels further strengthen these classes.
  - The initial bias can reinforce itself.



Adaptive thresholds require meaningful prediction statistics for every class.

Self-Adaptive Fairness

$$\mathcal{L}_f = -\mathcal{H}\left(\text{SumNorm}\left(\frac{\tilde{p}_t}{\tilde{h}_t}\right), \text{SumNorm}\left(\frac{\bar{p}}{\bar{h}}\right)\right),$$

$$\mathcal{L} = \mathcal{L}_s + w_u\mathcal{L}_u + w_f\mathcal{L}_f,$$

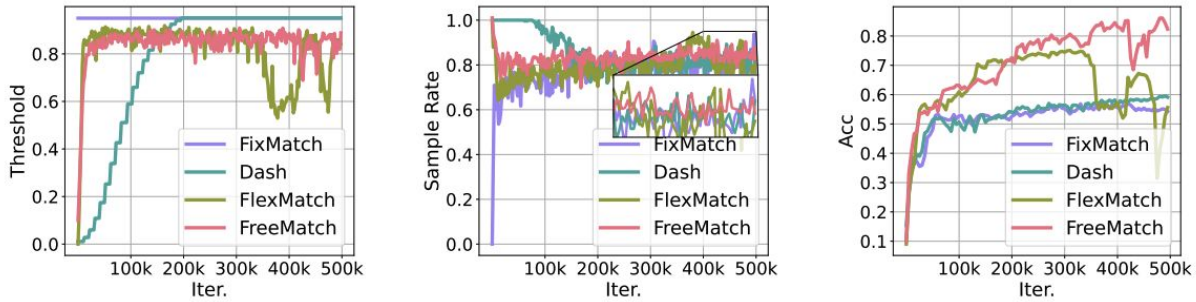
- $\tilde{p}_t$: EMA of predicted class probabilities
- $\tilde{h}_t$: EMA of pseudo-label histogram
- $\bar{p}$: predictions selected in the current minibatch
- $\bar{h}$: pseudo-label histogram of the current minibatch
- SumNorm: normalize values so that they sum to one

- $\mathcal{L}_s$: labeled-data loss
- $\mathcal{L}_u$: adaptive pseudo-label loss
- $\mathcal{L}_f$: self-adaptive fairness loss

SAT selects samples. SAF prevents the selection process from collapsing toward a few classes.

How Does the Threshold Change During Training?

- Early training
  - FreeMatch uses a lower threshold.
  - More unlabeled samples enter training.
  - Accuracy improves faster.
- Later training
  - The threshold increases with confidence.
  - Incorrect pseudo-labels are filtered.
  - The sampling rate remains relatively high.



(a) Confidence threshold (b) Sampling rate (c) Accuracy

FreeMatch changes both the quality and quantity of pseudo-labels during training.

Does Adaptive Thresholding Matter?

- Global and local adaptation work better together.
- FreeMatch improves accuracy without additional model inference.
- The gain is especially large when labeled data are extremely limited.

Table 3: Comparison of different thresholding schemes.

Threshold	CIFAR-10 (40)
τ (FixMatch)	7.47±0.28
$\tau * \mathcal{M}(\beta(c))$ (FlexMatch)	4.97±0.06
$\tau * \text{MaxNorm}(\tilde{p}_t(c))$	5.13±0.03
τ_t (Global)	6.06±0.65
$\tau_t * \mathcal{M}(\beta(c))$	8.40±2.49
CBST	16.65±2.90
RT (AdaMatch)	6.09±0.65
SAT (Global and Local)	**4.92**±0.04

Table 2: Error rates and runtime on ImageNet with 100 labels per class.

	Top-1	Top-5	Runtime (sec./iter.)
FixMatch	43.66	21.80	**0.4**
FlexMatch	41.85	19.48	0.6
FreeMatch	**40.57**	**18.77**	**0.4**

The threshold becomes part of the learning process, rather than a fixed hyperparameter.

Are low-confidence unlabeled samples useless?

AllMatch: Exploiting All Unlabeled Data for Semi-Supervised Learning

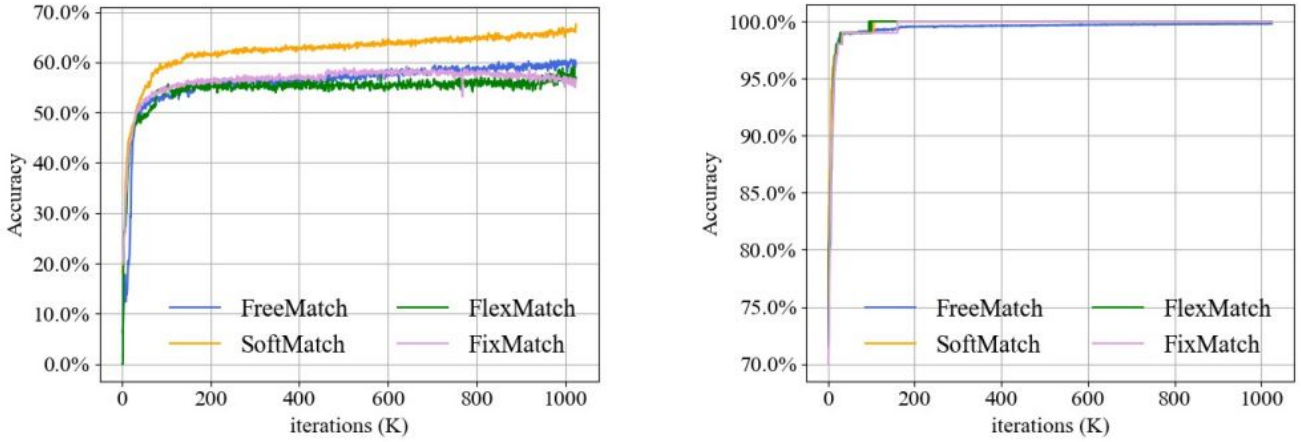
Zhiyu Wu , Jinshi Cui*

National Key Laboratory of General Artificial Intelligence,
School of Intelligence Science and Technology, Peking University
wuzhiyu@pku.edu.cn, cjs@cis.pku.edu.cn

Wu, et. al., IJCAI 2024, AllMatch: Exploiting All Unlabeled Data for Semi-Supervised Learning

Are Low-Confidence Pseudo-Labels Completely Useless?

- Threshold-based methods divide unlabeled samples into two groups.
  - High confidence
    - Use the top-1 prediction as a pseudo-label.
  - Low confidence
    - Discard the sample.
    - Provide no training signal.



(a) Dropped pseudo-label *acc*. (b) Top-5 *acc* of pseudo-labels.

The proportion of pseudo-labels discarded (falling below the threshold) that are actually correct in their top-1/5 prediction.

Low confidence does not mean no semantic information.

From FreeMatch to AllMatch

- AllMatch contains two components.
  - Class-Specific Adaptive Threshold, CAT
    - Estimates global learning progress
    - Estimates class-specific learning progress
    - Selects reliable top-1 pseudo-labels
  - Binary Classification Consistency, BCC
    - Divides classes into candidate and negative groups
    - Uses low-confidence samples
    - Enforces the same division across augmentations

AllMatch aims to provide learning signals for all unlabeled samples by combining CAT and BCC. CAT improves pseudo-label selection. BCC learns from the remaining samples.

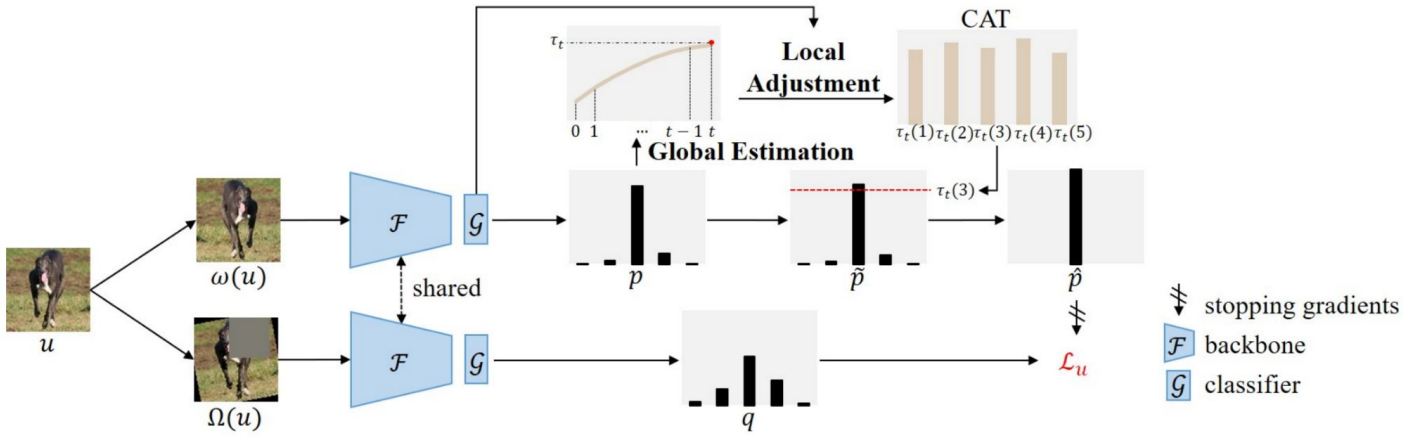
From FreeMatch to AllMatch

FreeMatch	AllMatch
Adaptive global threshold	Adaptive global threshold
Prediction-based local threshold	Classifier-weight-based local threshold
Discards low-confidence samples	Uses every unlabeled sample
Top-1 pseudo-label	Candidate versus negative classes

Class-Specific Adaptive Threshold

- Global estimation
  - Uses the average confidence of unlabeled samples
  - Tracks the overall learning progress
- Local adjustment
  - Uses the norm of each classifier weight
  - Estimates how well each class has been learned

$$\tau_t = \begin{cases} \frac{1}{C} & if\ t = 0 \\ m\tau_{t-1} + (1-m)\frac{1}{B_U}\sum_{i=1}^{B_U} max(p_i) & otherwise \end{cases}$$



The global threshold is the EMA of the average confidence on unlabeled data.
The main difference from FreeMatch lies in the local adjustment.

Classifier Weights as Learning-Status Indicators

- A larger classifier-weight norm tends to produce larger logits.
- Classes with more confident samples tend to have larger weight norms.
- The weight norm reflects the learning status of each class.
- Difficult classes receive lower thresholds.

$$z_c = f W_c^\top$$

$$z_c = \|f\| \, \|W_c\| \cos\theta_c$$

- f: feature of an input image
- W_c: classifier weight for class c
- θ_c: angle between f and W_c
- z_c: logit for class c

$$\tau_t(c) = \tau_t \cdot \frac{||W_c||}{max\{||W_c|| : c \in [1, \cdots, C]\}}$$

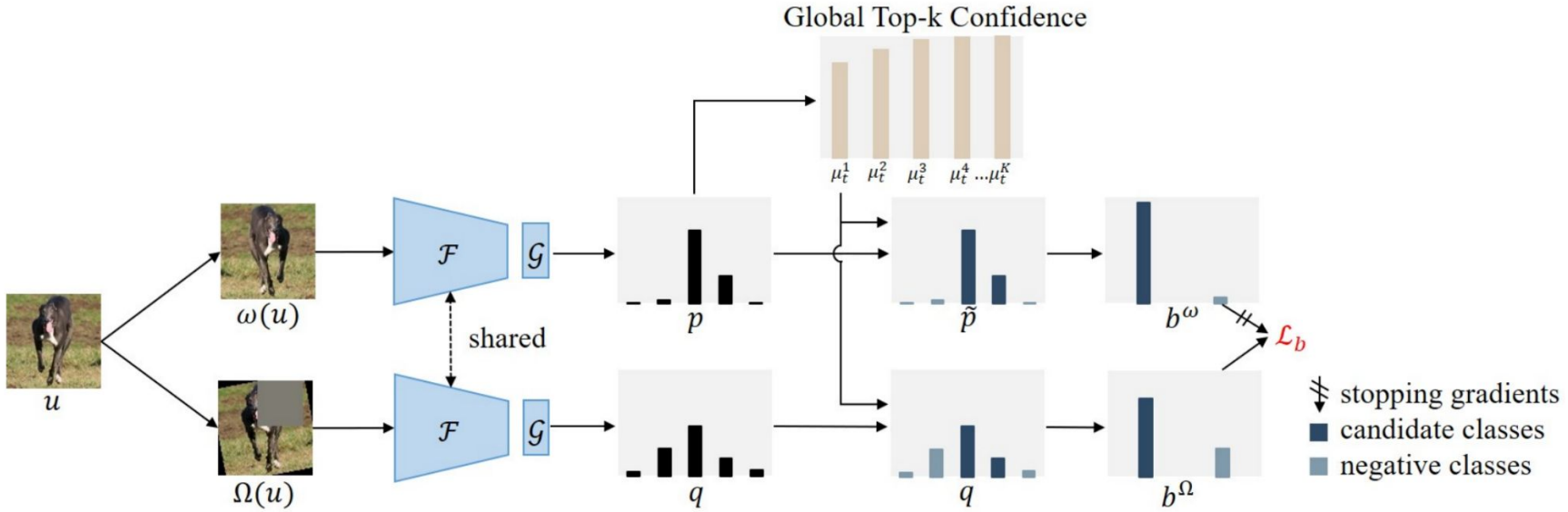
The classifier itself provides evidence about class-specific learning progress.

Binary Classification Consistency

- When top-1 confidence is insufficient:
  - Select several high-ranked classes as candidates.
  - Treat the remaining classes as negative options.
  - Convert the multi-class prediction into a binary prediction.
  - Match the binary prediction across weak and strong views.

$$b_i = \left[\sum_{c \in \mathcal{C}_i} p_i(c), \sum_{c \in \mathcal{N}_i} p_i(c) \right]$$

positive (high-ranked) negative (remaining)



How Many Candidate Classes?

- The number of candidate classes k is not set to the same value for all samples.
  - This is the cumulative probability assigned to the top k classes for sample i.

$$p_i^k = \sum_{j=1}^{k} p_{i,c_j} \quad (p_{i,c_1} \geq p_{i,c_2} \geq \cdots)$$

  - This indicates how much probability the overall model typically assigns to the top k classes.

$$\mu_t^k = \begin{cases} \frac{k}{C} & if\, t = 0 \\ m\mu_{t-1}^k + (1-m)\frac{1}{B_U}\sum_{i=1}^{B_U} p_i^k & otherwise \end{cases}$$

How Many Candidate Classes?

- Confident samples use one candidate class.
- Ambiguous samples use multiple candidate classes.
- The candidate-set size depends on each sample.
- K prevents the candidate set from becoming trivial.

$$k_i = \begin{cases} 1 & \text{if } \lambda(\tilde{p}_i) = 1 \\ min(min\{k : \tilde{p}_i^k \geq \mu_t^k\}, K) & otherwise \end{cases}$$

More ambiguous samples receive larger candidate sets.

Consistency Between Candidate and Negative Classes

- Candidate classes are determined from the weak view.
- The same candidate-negative division is applied to the strong view.
- The strong view should preserve the binary semantic decision.
- BCC is applied to every unlabeled sample.

$$b_i^{\omega} = [\sum_{j=1}^{k_i} \tilde{p}_{i,c_j}, \sum_{j=k_i+1}^{C} \tilde{p}_{i,c_j}] \quad (\tilde{p}_{i,c_1} \geq \tilde{p}_{i,c_2} \geq \cdots) \qquad \mathcal{L}_b = \frac{1}{B_U} \sum_{i=1}^{B_U} \mathcal{H}(b_i^{\omega}, b_i^{\Omega})$$

$$b_i^{\Omega} = [\sum_{j=1}^{k_i} q_{i,c_j}, \sum_{j=k_i+1}^{C} q_{i,c_j}]$$

- b_i^{ω}: binary prediction of the weak view
- b_i^{Ω}: binary prediction of the strong view
- First entry: probability mass of candidate classes
- Second entry: probability mass of negative classes

weak candidate membership = strong candidate membership
Uncertain samples receive coarse but reliable supervision

Two Levels of Supervision

High-confidence sample

$$\max(\tilde{p}_i) \geq \tau_t(c_i)$$

- Candidate set size: $k_i = 1$
- Standard pseudo-label consistency
- Fine class-level supervision

Low-confidence sample

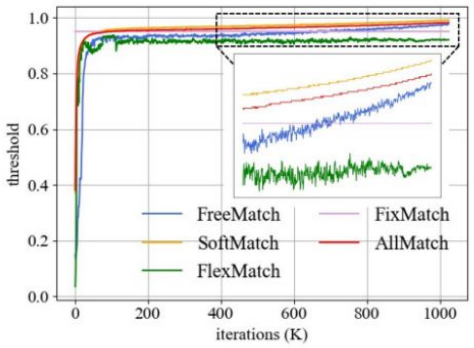
$$\max(\tilde{p}_i) < \tau_t(c_i)$$

- Candidate set size: $k_i > 1$
- Binary classification consistency
- Coarser semantic supervision

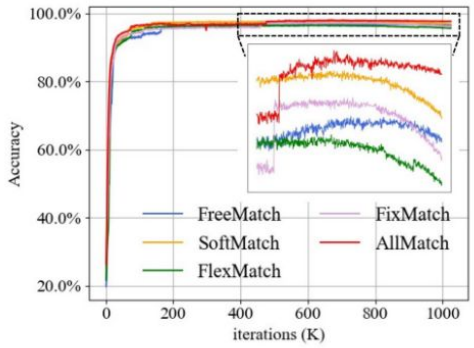
AllMatch changes the strength of supervision rather than discarding uncertain samples.

Does AllMatch Really Use All Samples?

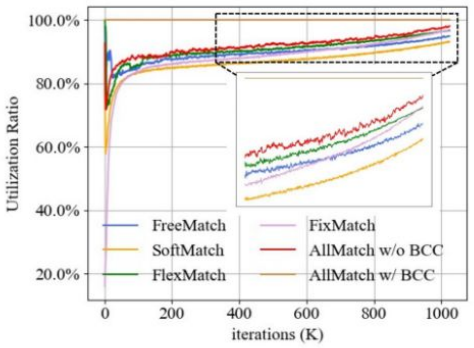
- CAT
  - Improves selected pseudo-label accuracy
  - Maintains a high utilization ratio
- BCC
  - Provides supervision for rejected samples
  - Reaches a 100% unlabeled-data utilization ratio
  - Candidate-negative accuracy exceeds top-1 accuracy



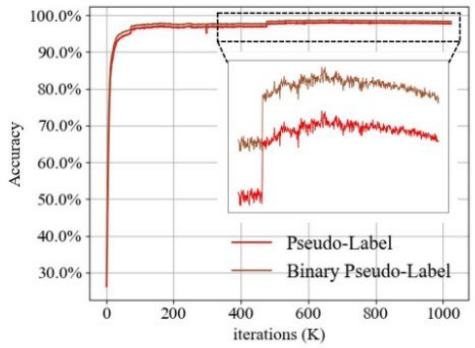
(a) Class-average threshold.



(b) Selected pseudo-label *acc*.



(c) Utilization ratio.



(d) Binary pseudo-label *acc*.

Ablation Study

- Global adaptive thresholding provides the largest initial gain.
- Class-specific adjustment further improves difficult classes.
- BCC consistently improves the complete CAT model.
- The gains are largest in extremely low-label settings.

GE	LA	BCC	CIFAR-10-10	CIFAR-100-400	STL-10-40	CIFAR-10-LT-150
			82.09	53.58	64.03	70.38
✓			89.44	61.58	80.48	71.58
✓	✓		92.66	63.04	87.43	73.48
✓	✓	✓	94.91	63.56	88.14	74.25

Table 4: Ablation study (%). (GE: global estimation in CAT. LA: local adjustment in CAT. BCC: binary classification consistency.)

GE: global estimation
LA: local adjustment
BCC: binary classification consistency

Comparison with Recent SSL Methods

- AllMatch consistently improves over FreeMatch.
- The largest gain appears when labels are extremely scarce.
- Low-confidence samples are most useful in difficult SSL settings.

Datasets	CIFAR-10				CIFAR-100			SVHN		STL-10	
# Label	10	40	250	4000	400	2500	10000	40	1000	40	1000
MixMatch [Berthelot *et al.*, 2019b]	34.24±7.06	63.81±6.48	86.37±0.59	93.34±0.26	32.41±0.66	60.24±0.48	72.22±0.29	69.40±8.39	96.31±0.37	45.07±0.96	78.30±0.68
ReMixMatch [Berthelot *et al.*, 2019a]	79.23±7.48	90.12±1.03	93.70±0.05	95.16±0.01	57.25±1.05	73.97±0.35	**79.98±0.27**	75.96±9.13	94.84±0.31	67.88±6.24	93.26±0.14
UDA [Xie *et al.*, 2020]	65.47±10.69	89.38±3.75	94.84±0.06	95.71±0.07	53.61±1.59	72.27±0.21	77.51±0.23	94.88±4.27	98.11±0.01	62.58±8.44	93.36±0.17
FixMatch [Sohn *et al.*, 2020]	82.09±2.48	92.53±0.28	95.14±0.05	95.79±0.08	53.58±0.82	71.97±0.16	77.80±0.12	96.19±1.18	98.04±0.03	64.03±4.14	93.75±0.33
Dash [Xu *et al.*, 2021]	73.72±14.09	91.07±3.11	94.84±0.23	95.64±0.11	55.18±0.96	72.85±0.22	78.12±0.07	97.81±0.18	98.03±0.01	65.48±4.30	93.61±0.56
MPL [Pham *et al.*, 2021]	76.45±6.01	93.38±0.91	94.24±0.24	95.45±0.04	53.74±1.84	72.29±0.19	78.26±0.09	90.67±8.02	97.72±0.02	64.24±4.83	93.34±0.08
FullMatch [Chen *et al.*, 2023b]	-	94.11±1.01	**95.36±0.12**	**96.25±0.08**	59.42±1.40	73.06±0.40	78.56±0.10	97.65±0.10	98.01±0.03	-	94.26±0.09
FlexMatch [Zhang *et al.*, 2021]	90.02±1.95	95.03±0.06	95.02±0.09	95.81±0.01	60.06±1.62	73.51±0.20	78.10±0.15	91.81±3.20	93.28±0.30	76.65±2.07	94.23±0.18
SoftMatch [Chen *et al.*, 2023a]	93.04±1.27	95.09±0.12	95.18±0.09	95.96±0.02	62.90±0.77	73.34±0.25	77.97±0.03	97.67±0.25	97.99±0.01	85.28±1.93	94.27±0.24
FreeMatch [Wang *et al.*, 2022]	92.09±0.85	95.10±0.04	95.12±0.18	95.90±0.02	62.02±0.42	73.53±0.20	78.32±0.03	**98.03±0.02**	98.04±0.03	84.68±0.82	94.37±0.15
AllMatch	**94.91±0.27**	**95.20±0.08**	95.28±0.06	96.14±0.04	**63.56±0.78**	**74.16±0.12**	78.66±0.09	97.56±0.12	**98.15±0.06**	**88.14±1.22**	**94.91±0.26**

Table 1: Top-1 accuracy (%) on CIFAR-10, CIFAR-100, SVHN, and STL-10 with varying number of labeled samples. **Bold** indicates the best performance, and underline denotes the second best performance.

A low-confidence sample may not provide a class label, but it can still provide class-exclusion information.

- FreeMatch
  - How should we select reliable pseudo-labels?
    - Adaptive threshold
- AllMatch
  - What should we do with the samples that are not selected?
    - Candidate-negative consistency

Should Every Pseudo-Label Face the Same Level of Perturbation?

(Pseudo-Labeling with Consistency Regularization)

Image-Feature Weak-to-Strong Consistency: An Enhanced Paradigm for Semi-Supervised Learning

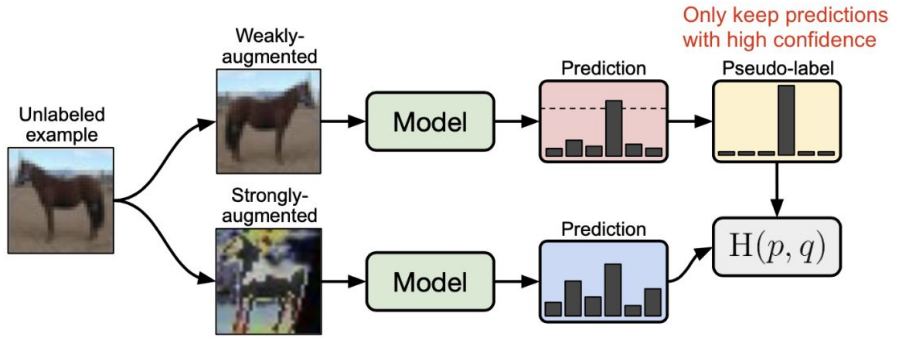
Zhiyu Wu and Jinshi Cui

National Key Laboratory of General Artificial Intelligence,
School of Intelligence Science and Technology, Peking University
wuzhiyu@pku.edu.cn, cjs@cis.pku.edu.cn

Wu, et. al., ECCV 2024, Image-Feature Weak-to-Strong Consistency: An Enhanced Paradigm for Semi-Supervised Learning

Should Every Pseudo-Label Face the Same Level of Perturbation?

- Pseudo-labeling with consistency regularization
  - A weakly augmented view generates a pseudo-label.
  - A strongly augmented view is trained to match it.
  - The same consistency task is applied to every selected sample.
- In the original FixMatch,
  - The same strong augmentation is applied to all samples that exceed the confidence threshold.
  - However, surpassing the threshold does not mean that all such samples possess the same level of difficulty.



But are all selected samples equally difficult to learn?

Image-Level Weak-to-Strong Consistency

- The threshold decides whether to use a pseudo-label, but not how much perturbation the sample needs.

$$\mathcal{L}_u = \frac{1}{B_U} \sum_{i=1}^{B_U} \mathbb{1}(\max(p_i^{\mathcal{I}_w}) \geq \tau_t) \mathcal{H}(\hat{p}_i^{\mathcal{I}_w}, p_i^{\mathcal{I}_s})$$

- $p_i^{\mathcal{I}_w}$: prediction from the weakly augmented image
- $\hat{p}_i^{\mathcal{I}_w}$: one-hot pseudo-label
- $p_i^{\mathcal{I}_s}$: prediction from the strongly augmented image
- τ_t: pseudo-label selection threshold
- $\mathcal{H}$: cross-entropy consistency loss

The Problem: Too Many Naive Samples

- Naive sample
  - The model predicts the pseudo-label correctly even after strong augmentation.
  - Its confidence remains high.
  - Its consistency loss becomes nearly zero.

$$\mathcal{H}\left(\hat{p}_i^{\mathcal{I}_w}, p_i^{\mathcal{I}_s}\right) \approx 0$$

- Challenging sample
  - Strong augmentation already makes prediction difficult.
  - Additional perturbation may destroy useful information.
  - Its loss is already sufficiently large.

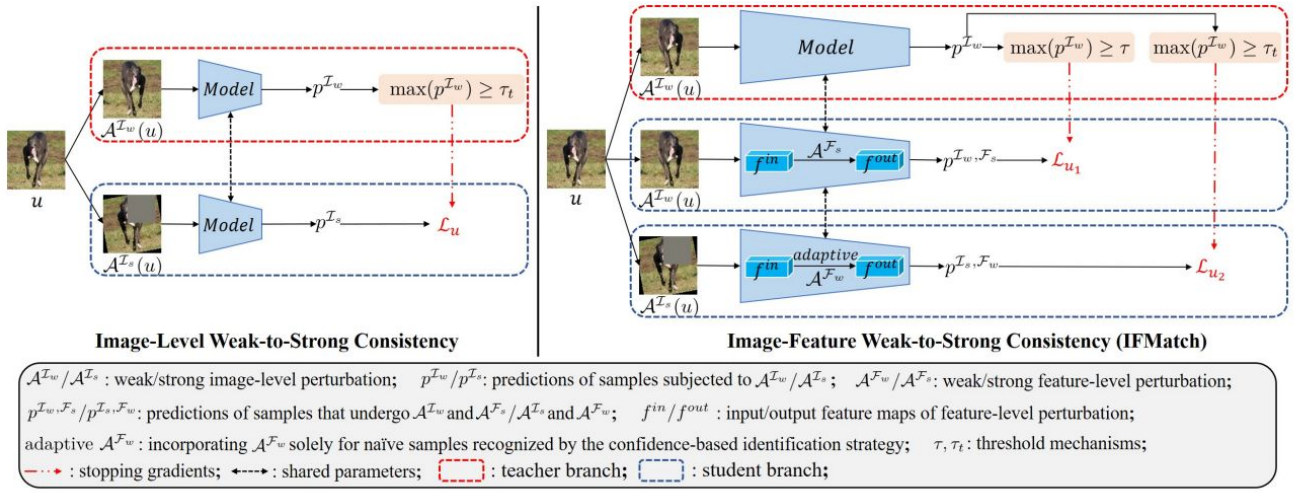
Naive samples need a stronger task, while challenging samples do not.

IFMatch: Image-Feature Weak-to-Strong Consistency

- Teacher branch
  - Generates the pseudo-label.
- Student Branch I
  - Weak image perturbation + strong feature perturbation.
- Student Branch II
  - Strong image perturbation + optional weak feature perturbation.

Conventional image-level consistency

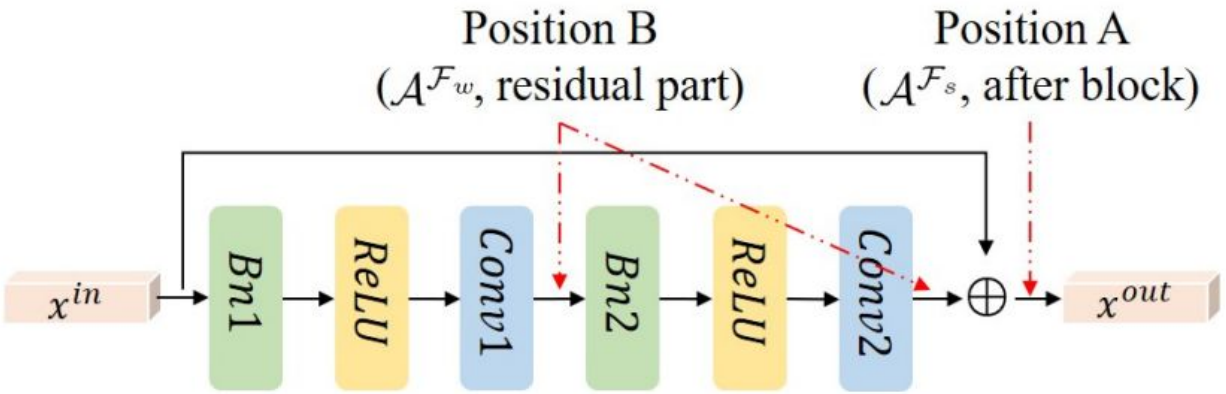
IFMatch



IFMatch expands consistency regularization from the image space to the feature space.
Not to replace image perturbation with feature perturbation, but rather to incorporate both types of perturbation

Weak and Strong Feature Perturbations

- Position A: Strong Feature Perturbation
  - Applied **after** the **residual** block.
  - **Directly** perturbs the block output.
  - **Every** subsequent layer **receives** the **perturbed** feature.
  - Produces a relatively **strong** effect.
- Position B: Weak Feature Perturbation
  - Applied **inside** the **residual** branch.
  - The **identity** connection remains **unchanged**.
  - Its effect is **partially moderated** by the residual connection.
  - Produces a relatively **weak** effect.



Weak and Strong Feature Perturbations

- Perturbation strength is controlled by where the feature is modified.

Type	Feature-level operation
Movement	Translation, shearing
Dropout	Channel-wise, spatial-wise dropout
Value	Local smoothing using random convolution

Two Complementary Consistency Branches

- Branch I: Feature-Level Consistency
  - Weak image + strong feature perturbation
    - Focuses on feature-level consistency.
    - Uses a **high** fixed threshold
    - Only highly reliable pseudo-labels supervise strong feature perturbation.
- Branch II: Image-Level Consistency
  - Strong image + conditional weak feature perturbation
    - Retains conventional image-level consistency.
    - Uses the threshold mechanism of the base method.
    - Weak feature perturbation is added only when appropriate.

$$\mathcal{L}_{u_1} = \frac{1}{B_U} \sum_{i=1}^{B_U} \mathbb{1}(\max(p_i^{\mathcal{I}_w}) \geq \tau) \mathcal{H}(\hat{p}_i^{\mathcal{I}_w}, p_i^{\mathcal{I}_w, \mathcal{F}_s}) \quad \mathcal{L}_{u_2} = \frac{1}{B_U} \sum_{i=1}^{B_U} \mathbb{1}(\max(p_i^{\mathcal{I}_w}) \geq \tau_t) \mathcal{H}(\hat{p}_i^{\mathcal{I}_w}, p_i^{\mathcal{I}_s, \mathcal{F}_w})$$

Both branches use the same pseudo-label, but impose different consistency tasks.

Why Not Simply Combine Strong Perturbations?

- More perturbation is not always better.
- Strong image and strong feature perturbations can destroy the semantic signal.
- Even weak feature perturbation is harmful when applied to every challenging sample.
- Confidence-based identification is therefore required.

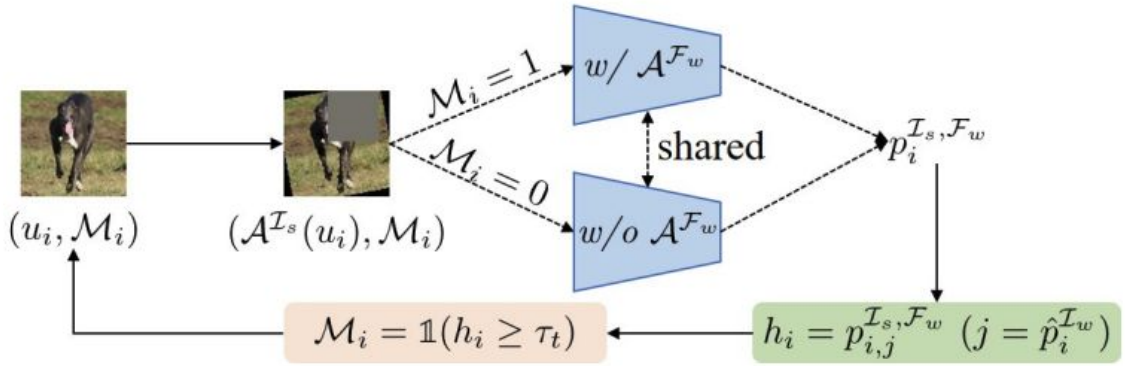
Table 3: Experiments (%) on the strategies for integrating image-level and feature-level perturbations. CBI abbreviates the confidence-based identification strategy. CIFAR-m-n denotes CIFAR-m with n labeled samples. Line 3 combines the structure setting of UniMatch [40] and our proposed feature-level perturbation techniques.

Branch I	Branch II	CIFAR-10-40	CIFAR-100-400
FixMatch (baseline, no $\mathcal{A}^{\mathcal{F}}$)		92.53	53.58
$\mathcal{A}^{\mathcal{F}s} + \mathcal{A}^{\mathcal{I}s}$ in one branch		88.26	51.47
$\mathcal{A}^{\mathcal{F}s}$	$\mathcal{A}^{\mathcal{I}s}$	95.53	64.54
$\mathcal{A}^{\mathcal{F}s} + \mathcal{A}^{\mathcal{I}w}$	$\mathcal{A}^{\mathcal{F}w} + \mathcal{A}^{\mathcal{I}s}$	95.47	63.56
$\mathcal{A}^{\mathcal{F}s} + \mathcal{A}^{\mathcal{I}w}$	$\mathcal{A}^{\mathcal{F}w} + \mathcal{A}^{\mathcal{I}s}$ + CBI	95.82	66.26

Improved performance is not simply applying a large amount of perturbation
But rather adjusting the intensity and tailoring it to each individual sample.

Confidence-Based Identification

- Step 1. Measure Target Confidence
- Step 2. Identify the Sample
- Step 3. Assign Perturbations



$$h_i = p_{i,j}^{\mathcal{I}_s,\mathcal{F}_w}\ (j = \hat{p}_i^{\mathcal{I}_w})$$

$$\mathcal{M}_i = \mathbb{1}(h_i \geq \tau_t)$$

$$\text{Perturbations for } u_i = \begin{cases} \{\mathcal{A}^{\mathcal{I}_s}, \mathcal{A}^{\mathcal{F}_w}\} & \mathcal{M}_i = 1 \\ \{\mathcal{A}^{\mathcal{I}_s}\} & \mathcal{M}_i = 0 \end{cases}$$

If confidence remains high after strong image augmentation:
→ The sample is still considered "easy."
→ Apply weak feature perturbation.
If confidence becomes low after strong image augmentation:
→ The sample is already considered "difficult."
→ Do not apply feature perturbation.

The process of deciding whether to add additional feature perturbations

Different Branches Prefer Different Thresholds

- Branch I
  - Strong feature perturbation can severely change the representation.
  - Incorrect pseudo-labels are especially harmful.
  - Therefore, it uses a conservative fixed threshold
- Branch II
  - Mainly follows the existing image-level consistency branch.
  - It inherits the dynamic threshold of FlexMatch, SoftMatch, or FreeMatch.
  - The threshold balances pseudo-label quality and utilization.

Table 5: Exploration (%) for the threshold strategy in the first branch.

Branch I	Branch II	CIFAR-10-40	CIFAR-100-400
FlexMatch $\tau = 0.95$	FlexMatch	95.28	63.38
	FlexMatch	**95.90**	**66.28**
SoftMatch $\tau = 0.95$	SoftMatch	95.39	63.80
	SoftMatch	**95.84**	**67.02**
FreeMatch $\tau = 0.95$	FreeMatch	95.56	64.45
	FreeMatch	**95.83**	**66.65**

The appropriate threshold depends on the consistency task, not only on the dataset.

IFMatch Improves Existing Pseudo-Labeling Methods

- IFMatch is not a new pseudo-label generation method.
- It can be applied to different thresholding methods.
- It improves how selected pseudo-labels are used for consistency learning.
- The benefit is especially large under scarce-label settings.

Datasets	CIFAR-10			CIFAR-100			SVHN		STL-10		ImageNet	
# Label	40	250	4000	400	2500	10000	40	1000	40	1000	100k	400k
VAT [25]	25.34	58.97	89.49	14.80	53.16	67.86	25.25	95.89	25.26	62.05	-	-
MeanTeacher [33]	29.91	62.54	91.90	18.89	54.83	68.25	63.91	96.73	28.28	66.10	-	-
MixMatch [4]	63.81	86.37	93.34	32.41	60.24	72.22	69.40	96.31	45.07	78.30	-	-
ReMixMatch [3]	90.12	93.70	95.16	57.25	73.97	79.98	75.96	94.84	67.88	93.26	-	-
UDA [38]	89.38	94.84	95.71	53.61	72.27	77.51	94.88	98.11	62.58	93.36	-	-
Dash [39]	91.07	94.84	95.64	55.18	72.85	78.12	97.81	98.03	65.48	93.61	-	-
MPL [29]	93.38	94.24	95.45	53.74	72.29	78.26	90.67	97.72	64.24	93.34	-	-
FixMatch [32]	92.53	95.14	95.79	53.58	71.97	77.80	96.19	98.04	64.03	93.75	56.34	67.72
IFMatch (Fix)	95.82	95.97	**96.49**	66.26	74.65	80.01	97.75	98.15	78.54	95.28	61.26	71.48
FlexMatch [42]	95.03	95.02	95.81	60.06	73.51	78.10	91.81	93.28	70.85	94.23	58.15	68.69
IFMatch (Flex)	**95.90**	95.99	96.42	66.28	75.03	79.87	97.76	98.04	79.83	95.24	61.75	71.14
SoftMatch [5]	95.09	95.18	95.96	62.90	73.34	77.97	97.67	97.99	78.58	94.27	59.43	70.51
IFMatch (Soft)	95.84	95.93	96.36	**67.02**	74.99	79.96	98.04	**98.22**	82.46	95.22	**62.44**	**71.66**
FreeMatch [35]	95.10	95.12	95.90	62.02	73.53	78.32	98.03	98.04	84.44	94.37	58.48	69.48
IFMatch (Free)	95.83	**96.02**	96.44	66.65	**75.16**	**80.08**	**98.08**	98.18	**84.75**	**95.29**	61.69	71.30

Pseudo-label confidence determines whether a target is reliable.
Target confidence determines how strongly the sample should be perturbed.

From Selecting to Using Pseudo-Labels

	FreeMatch	AllMatch	IFMatch
Question	Which pseudo-labels should be selected?	Can low-confidence predictions still be useful?	Should every pseudo-label face the same perturbation?
Problem	A fixed threshold ignores the learning status.	Low-confidence samples are discarded.	Selected samples have different learning difficulties.
Key idea	Self-adaptive global and class-specific thresholds	Candidate-negative consistency for low-confidence samples	Confidence-aware image-feature perturbation
Supervision	Hard pseudo-label	Fine or coarse pseudo-label	Pseudo-label with adaptive consistency difficulty

Modern pseudo-labeling is not only about generating labels.
It decides which information to trust, how much information to use, and how strongly each sample should be perturbed.
The key question has shifted from “What is the pseudo-label?” to “How should its information be used?”

Next Week..

- Weakly supervised ..